

VAULT-TEC

MEDIA OPTIMIZER

Documentation complète d'installation et d'administration

Complete installation & administration guide

Version 1.8.2 — Extension MediaWiki — Archives de Vault-Tec

Document **bilingue FR / EN** · moteur libvips · GIF (gifsicle) · AVIF expérimental · dépendances · benchmarks

FR · Partie française ci-dessous · **EN** · English version follows

1. Présentation
2. Prérequis & dépendances — 2.5 installation des dépendances (nouveau)
3. Installation
4. Configuration — 4.6 GIF/gifsicle, 4.7 libvips, 4.8 mode gros wiki, 4.9 AVIF (nouveaux)
5. Architecture technique
6. Pages spéciales
7. Installation des binaires de second passage
8. Maintenance — 8.6 backfill en ligne de commande (nouveau)
9. Dépannage
10. Référence rapide
11. Foire aux questions
12. Désinstallation
13. Sécurité
14. Glossaire
15. Tutoriel pas-à-pas (de zéro à la production)
16. Benchmarks — coût / bénéfice (nouveau)

Nouveautés depuis la v1.4.1 (jusqu'à la 1.8.1). Ce manuel reprend l'intégralité de la documentation 1.4.1 et y intègre tout ce qui a été ajouté depuis : **moteur d'image libvips** optionnel (§4.7), **optimisation GIF sans perte** via gifsicle (§4.6), **mode « gros wiki »** en ligne de commande (§4.8 et §8.6), **AVIF expérimental** (§4.9), l'**installation des dépendances** (§2.5), une section **benchmarks coût/bénéfice** mesurés et honnêtes (§16), et les **correctifs 1.8.1** (resynchronisation des métadonnées après la 1^{re} passe ; GIF animés jamais figés sous GD). Les éléments expérimentaux sont signalés **EXPÉRIMENTAL**, les ajouts **NOUVEAU**.

1. Présentation

Vault-Tec Media Optimizer est une extension MediaWiki qui réduit le poids des médias d'un wiki de deux manières complémentaires : elle optimise les fichiers d'origine (PNG, JPEG, GIF) de façon sans perte, et elle génère pour chaque image une version WebP (et, en option expérimentale, AVIF) servie automatiquement aux navigateurs via une balise `<picture>`. Les visiteurs reçoivent ainsi des images nettement plus légères sans aucune perte de qualité visible, ce qui accélère le chargement des pages et réduit la bande passante consommée.

L'extension est conçue pour fonctionner de manière transparente : les nouveaux fichiers téléversés sont traités automatiquement, et un mécanisme de rattrapage (« backfill ») permet de traiter l'intégralité des fichiers déjà présents. Trois pages spéciales offrent un diagnostic complet, le pilotage du rattrapage et des statistiques détaillées.

1.1 Principe de fonctionnement

Le cœur de l'extension repose sur une distinction essentielle entre deux notions :

- **L'original optimisé** : le fichier source (PNG/JPEG/GIF) est recompressé sans perte. Les pixels restent strictement identiques ; seule la taille du fichier sur le disque diminue. Ce gain concerne l'espace de stockage du serveur.
- **La version WebP** : une copie au format WebP est générée et stockée séparément. C'est cette version, beaucoup plus légère, qui est envoyée aux navigateurs compatibles. Ce gain concerne la bande passante et la vitesse de chargement pour les visiteurs.

La balise `<picture>` insérée dans le HTML permet au navigateur de choisir automatiquement : WebP (ou AVIF) s'il le supporte (tous les navigateurs modernes), format d'origine sinon. L'image originale reste donc toujours accessible, ce qui garantit la compatibilité totale et le bon fonctionnement de fonctionnalités comme la visionneuse multimédia.

1.2 Ce que l'extension ne fait pas

Pour éviter toute confusion, il est important de comprendre les limites volontaires de l'extension :

- Elle n'altère jamais la qualité visuelle des images d'origine (optimisation strictement sans perte pour les PNG ; ré-encodage à haute qualité pour les JPEG).
- Elle ne supprime pas les fichiers originaux : ils restent en place et accessibles.
- Pour les **GIF**, l'optimisation de l'original est désormais possible et **strictement sans perte** via `gifsicle` (§4.6), sûre pour l'animation et **désactivée par défaut**. Sans `gifsicle`, l'original du GIF n'est pas modifié (pour ne prendre aucun risque sur l'animation) et seule sa version WebP est produite — le comportement historique.
- Elle ne dépend d'aucun service externe ou cloud : tout le traitement se fait sur votre serveur.

2. Prérequis & dépendances

L'extension nécessite un environnement répondant aux conditions suivantes. La page spéciale **VaultTec_État** (décrite à la section 6.1) vérifie automatiquement l'ensemble de ces prérequis et signale tout problème.

2.1 Versions minimales

Composant	Version requise
MediaWiki	1.43 ou supérieur
PHP	8.1 ou supérieur
Bibliothèque d'images	Imagick (recommandé) ou GD, avec support WebP

2.2 Bibliothèque de traitement d'image

L'extension utilise en priorité **Imagick** (l'interface PHP vers ImageMagick), réputée plus fidèle et plus complète. En son absence, elle bascule automatiquement sur **GD**, présent par défaut dans la plupart des installations PHP. Dans les deux cas, le **support WebP** doit être activé dans la bibliothèque, sans quoi la génération WebP est impossible.

Conseil. Imagick est fortement recommandé. Il gère mieux la transparence des PNG, les profils colorimétriques et offre un contrôle plus fin de la compression (et un vrai WebP sans perte, là où GD ne fait qu'approximer). La page VaultTec_État indique clairement quelle bibliothèque est utilisée et si le support WebP est présent. *Voir aussi le moteur **libvips** (§4.7), alternative rapide et économe en mémoire à grande échelle.*

2.3 Système de fichiers

L'extension lit et écrit directement sur le système de fichiers local du wiki (backend de type `FSFileBackend`). Elle détecte les backends non locaux (comme Amazon S3 ou OpenStack Swift) et le signale, car le traitement en place n'y est pas possible sans adaptation. Le répertoire de destination des fichiers WebP (`images_webp/` par défaut) doit être accessible en écriture par l'utilisateur sous lequel tourne PHP.

2.4 Mémoire et temps d'exécution PHP

Le traitement d'images volumineuses peut être gourmand. Les valeurs recommandées :

- `memory_limit` : 256 Mo minimum. Une valeur de 128 Mo fonctionne pour la plupart des images mais peut échouer sur les très grandes. *Le moteur libvips réduit nettement ce besoin (§16.5).*
- `max_execution_time` : peu critique, car le gros du travail passe par la file de tâches (jobs) asynchrone et non par les requêtes web directes.

2.5 Installation des dépendances NOUVEAU

L'extension fonctionne avec un minimum : une bibliothèque d'image avec support WebP. Toutes les autres dépendances sont **optionnelles** et n'apportent qu'un bénéfice ciblé.

Dépendance	Rôle	Obligatoire ?
Imagick <i>ou</i> GD (avec WebP)	1 ^{re} passe + encodage WebP	Oui (l'une des deux)
libvips (<i>vips</i>)	Moteur rapide/léger à grande échelle (§4.7, §16)	Non — opt-in
<i>gifsicle</i>	1 ^{re} passe GIF sans perte, sûre pour l'animation (§4.6)	Non — gain GIF
<i>zopflipng</i>	2 ^e passe PNG sans perte, compression maximale (§4.4)	Non — gain disque
<i>oxipng</i>	2 ^e passe PNG sans perte, rapide (§4.4)	Non — gain disque
libheif+AV1 / <i>imageavif()</i>	AVIF expérimental (§4.9)	Non — expérimental

```
# Debian / Ubuntu – bibliothèque d'image (l'une) + binaires optionnels
sudo apt install php-imagick      # recommandé
sudo apt install php-gd           # repli, souvent déjà présent
sudo apt install gifsicle libvips-tools zopfli  # GIF + libvips + zopflipng
# RHEL / Alma / Rocky / Fedora (EPEL)
sudo dnf install php-imagick php-gd gifsicle vips-tools zopfli
# oxipng : souvent pas empaqueté -> compiler (évite les soucis de glibc)
cargo install oxipng
sudo systemctl restart php-fpm apache2 2>/dev/null || true
```

Vérifiez le **support WebP** de la bibliothèque (indispensable) :

```
php -r 'echo extension_loaded("imagick")&&in_array("WEBP",Imagick::queryFormats())?"Imagick+WebP OK\n":"";'
php -r '$i=gd_info();echo !empty($i["WebP Support"])?"GD+WebP OK\n":"GD sans WebP\n";'
```

Piège open_basedir (mutualisé / Plesk). Si PHP est restreint, un binaire installé dans `/usr/bin` reste invisible côté web même s'il fonctionne en ligne de commande. **Règle d'or :** copiez le binaire *dans* l'arborescence du wiki (par exemple `httpdocs/bin/`), faites `chmod 755`, donnez-lui pour propriétaire l'utilisateur PHP, puis configurez son **chemin absolu**. Cette règle vaut pour `vips`, `gifsicle`, `zopflipng` et `oxipng`.

3. Installation

3.1 Mise en place des fichiers

Décompressez l'archive de l'extension dans le répertoire `extensions/` de votre installation MediaWiki. Le dossier doit s'appeler exactement `VaultTecMediaOptimizer`, soit le chemin `extensions/VaultTecMediaOptimizer/`.

Important — transfert FTP. Si vous transférez les fichiers par FTP, utilisez impérativement le **mode binaire** et non le mode ASCII. Un transfert en mode ASCII corrompt notamment le fichier `extension.json` (modification des fins de ligne et de l'encodage), ce qui provoque l'erreur « is not a valid JSON file » au démarrage. En cas de doute, décompressez l'archive directement sur le serveur via SSH.

3.2 Activation dans LocalSettings.php

Ajoutez la ligne suivante à votre fichier `LocalSettings.php` :

```
wfLoadExtension( 'VaultTecMediaOptimizer' );
```

Les éventuels réglages personnalisés (voir section 4) doivent être placés **après** cette ligne, afin de ne pas être écrasés par les valeurs par défaut.

3.3 Mise à jour de la base de données

L'extension crée deux tables (`vtmo_image_optimization` et `vtmo_zopfli_thumb_stats`) et, lors d'une montée de version, ajoute une colonne. Exécutez le script de mise à jour de MediaWiki :

```
php maintenance/run.php update --quick
```

Sur les versions antérieures à MediaWiki 1.40, la commande équivalente est `php maintenance/update.php --quick`.

Note. Le script de mise à jour est idempotent : il peut être relancé sans risque. Il détecte ce qui existe déjà et n'applique que les changements nécessaires, qu'il s'agisse d'une installation neuve, d'une mise à jour depuis une version antérieure, ou d'une base déjà à jour.

3.3.1 En cas de blocage par une autre extension

Si le script `update.php` échoue à cause d'une autre extension (par exemple une extension dont le fichier SQL n'est pas idempotent), la migration de Vault-Tec Media Optimizer peut ne jamais être atteinte. Deux solutions :

1. Désactiver temporairement l'extension fautive dans `LocalSettings.php`, relancer `update.php`, puis la réactiver.
2. Créer manuellement les objets via SQL (à adapter au préfixe de tables si vous en utilisez un) :

```
ALTER TABLE vtmo_image_optimization
  ADD COLUMN IF NOT EXISTS io_png_zopfli_size INT UNSIGNED DEFAULT NULL;

CREATE TABLE IF NOT EXISTS vtmo_zopfli_thumb_stats (
  zts_id INT UNSIGNED NOT NULL,
  zts_thumb_count INT UNSIGNED DEFAULT 0 NOT NULL,
  zts_bytes_saved BIGINT UNSIGNED DEFAULT 0 NOT NULL,
  zts_updated_at BINARY(14) DEFAULT NULL,
  PRIMARY KEY(zts_id)
);
```

3.4 Vérification post-installation

Rendez-vous sur la page **Spécial:VaultTec_État**. Tous les contrôles doivent être au vert (ou orange pour les éléments optionnels non bloquants). C'est l'outil de référence pour confirmer que l'installation est saine avant de lancer le rattrapage.

4. Configuration

Tous les paramètres se définissent dans `LocalSettings.php`, après la ligne `wfLoadExtension`. Chaque paramètre possède une valeur par défaut raisonnable ; vous ne devez surcharger que ceux que vous souhaitez modifier. Le préfixe complet est `$wgVaultTecMediaOptimizer` (par exemple `$wgVaultTecMediaOptimizerEnabled`) ; il est abrégé en `...` dans les tableaux.

4.1 Paramètres généraux

Paramètre	Défaut	Description
<code>...Enabled</code>	<code>true</code>	Interrupteur principal. Mettre à <code>false</code> désactive entièrement l'extension.
<code>...Formats</code>	<code>png, jpeg, gif</code>	Types MIME traités. Tableau de chaînes (<code>image/png</code> , etc.).
<code>...MaxFileSize</code>	<code>52428800</code>	Taille maximale (en octets) d'un fichier à traiter. 50 Mio par défaut.
<code>...StripMetadata</code>	<code>true</code>	Supprime les métadonnées non essentielles (EXIF, etc.) tout en préservant le profil colorimétrique et la transparence.

4.2 Optimisation des originaux

Paramètre	Défaut	Description
<code>...OptimizeOriginals</code>	<code>true</code>	Active la recompression des fichiers d'origine. Si <code>false</code> , seuls les WebP sont générés (les originaux ne sont pas touchés).

L'optimisation des originaux dépend du type : les PNG sont recompressés strictement sans perte ; les JPEG sont optimisés (tables de Huffman, progressif, métadonnées retirées) mais via un ré-encodage Imagick qui n'est pas strictement sans perte (la qualité d'origine est préservée pour rendre la perte invisible) ; les GIF sont optimisés sans perte par `gifsicle` si vous l'activez (§4.6), sinon laissés intacts. Le tutoriel (section 15, étape 5) détaille ce point et explique comment exclure les JPEG si vous exigez une optimisation strictement sans perte.

Note. Désactiver `OptimizeOriginals` est utile si vous ne souhaitez pas modifier les fichiers d'origine sur le disque. Cela n'affecte en rien la génération WebP ni l'expérience des visiteurs, qui reçoivent toujours les versions WebP allégées.

4.3 Génération WebP

Paramètre	Défaut	Description
<code>...WebPDirectory</code>	<code>images_webp</code>	Répertoire (relatif à la racine du wiki) où sont stockés les fichiers WebP, séparément des originaux.
<code>...WebPQuality</code>	<code>85</code>	Qualité WebP pour les images avec perte (JPEG). Plage conseillée : 80-90.
<code>...WebPLosslessForPng</code>	<code>true</code>	Génère les WebP des PNG en mode sans perte, préservant la transparence.

4.4 Recompression PNG de second passage (zopflipng / oxipng)

Au-delà de l'optimisation de base, l'extension peut appliquer un second passage de recompression PNG sans perte à l'aide d'un outil externe. Deux moteurs sont disponibles, au choix. Ce second passage est **désactivé par défaut** et son seul bénéfice est un gain d'espace disque supplémentaire (de l'ordre de 8 à 20 % sur les PNG). Il n'améliore pas les WebP servis aux visiteurs : il n'a donc d'intérêt que si l'espace de stockage est une préoccupation.

4.4.1 Choix du moteur

- zopflipng** : compression maximale, mais lent (plusieurs secondes, voire minutes, par fichier volumineux). Idéal si la vitesse importe peu et qu'on veut gratter les derniers pourcents.
- oxipng** : optimiseur multi-threadé écrit en Rust, bien plus rapide (de l'ordre de la seconde par fichier), pour un résultat très proche (à quelques pourcents près). Recommandé pour le traitement de masse.

Paramètre	Défaut	Description
<code>...ZopfliEnabled</code>	<code>false</code>	Interrupteur principal du second passage, quel que soit le moteur.
<code>...PngEngine</code>	<code>zopflipng</code>	Moteur utilisé : <code>zopflipng</code> ou <code>oxipng</code> .
<code>...ZopfliBinary</code>	<code>zopflipng</code>	Chemin ou nom de commande du binaire <code>zopflipng</code> .
<code>...ZopfliIterations</code>	<code>15</code>	Nombre d'itérations <code>zopflipng</code> . Plus élevé = plus petit mais plus lent.
<code>...OxipngBinary</code>	<code>oxipng</code>	Chemin ou nom de commande du binaire <code>oxipng</code> .
<code>...OxipngLevel</code>	<code>2</code>	Niveau d'optimisation <code>oxipng</code> (0-6 ou <code>max</code>). 2 est un bon équilibre.

4.4.2 Exemple de configuration oxipng

Configuration type pour activer `oxipng` (voir section 7 pour l'installation du binaire) :

```
$wgVaultTecMediaOptimizerZopfliEnabled = true;
```



```
$wgVaultTecMediaOptimizerPngEngine = 'oxipng';
$wgVaultTecMediaOptimizerOxipngBinary = '/chemin/vers/votre/wiki/bin/oxipng';
```

4.5 Rattrapage (backfill)

Paramètre	Défaut	Description
... ProcessThumbnails	true	Recompresse aussi les miniatures à la volée lorsqu'elles sont générées.
... BackfillBatchSize	20	Nombre de fichiers traités par lot lors du rattrapage via la file de tâches.
... OnDemandThumbLimit	5	Nombre maximal de WebP de miniatures manquants générés <i>en synchrone</i> pendant un seul rendu de page (garde-fou de latence sur cache froid). 0 désactive la génération à la volée.

4.6 Optimisation GIF (gifsicle) NOUVEAU

Première passe GIF **strictement sans perte et sûre pour l'animation**, via le binaire externe `gifsicle`. L'extension ne passe que l'option `-0{niveau}` (optimisation structurelle : meilleur empaquetage LZW, transparence inter-frames, cadres à boîte englobante minimale) : jamais de redimensionnement, jamais d'altération des pixels, de la cadence ou du compteur de boucle. Le rendu de l'animation est donc identique au pixel près — seule sa représentation sur disque rétrécit. Elle tourne à l'upload **et** pendant le rattrapage CLI ; si `gifsicle` est absent ou désactivé, c'est un no-op inoffensif (la génération WebP se poursuit). Un garde-fou « ne garder que si plus petit » empêche tout gonflement.

Paramètre	Défaut	Description
...GifsicleEnabled	false	Active l'optimisation GIF sans perte.
...GifsicleBinary	gifsicle	Chemin ou nom de commande du binaire.
...GifsicleLevel	3	Niveau -0 (1-3) ; 3 essaie le plus de stratégies.

4.7 Moteur libvips (optionnel) NOUVEAU

Moteur d'image alternatif basé sur le binaire `vips` (aucune extension PHP requise). Il produit un WebP de **qualité/taille identique** à Imagick/GD (même bibliothèque libwebp), mais peut être **plus économe en mémoire**, en particulier sur les miniatures (voir \$16.5 pour des mesures honnêtes). Si le binaire est inutilisable, l'extension **retombe automatiquement** sur Imagick/GD.

Paramètre	Défaut	Description
...ImageEngine	auto	auto (Imagick puis GD) ou <code>vips</code> .
...VipsBinary	vips	Chemin ou nom de commande du binaire <code>vips</code> .

4.8 Mode « gros wiki » : file de tâches NOUVEAU

Paramètre	Défaut	Description
... UseJobQueue	true	false = ne plus enfiler de tâche à l'upload : on optimise alors les originaux en ligne de commande (\$8.6) à son propre rythme. Les miniatures reçoivent quand même leur WebP à la demande au rendu.

4.9 AVIF (expérimental) EXPÉRIMENTAL

Génère une copie **AVIF** en plus du WebP et la propose *avant* le WebP dans `<picture>` (les navigateurs compatibles choisissent l'AVIF, les autres retombent sur WebP puis sur l'original). Selon le contenu et l'encodeur, l'AVIF *peut* être plus compact que le WebP à qualité égale — mais pas toujours (sur du bruit ou en haute qualité il peut être plus gros). L'extension **compare donc l'AVIF au WebP et ne garde l'AVIF que s'il est strictement plus petit** ; sinon elle sert le WebP. L'AVIF exige un **encodeur AV1** (Imagick+libheif/ AV1, PHP-GD `imageavif()`, ou libvips `heifsave+AV1`) ; l'extension **sonde réellement** cette capacité et retombe proprement sur le WebP si elle manque. Encodage lent → désactivé par défaut.

Paramètre	Défaut	Description
...AvifEnabled	false	Active la génération AVIF (en plus du WebP).
... AvifDirectory	images_avif	Répertoire des copies AVIF (frère de <code>images/</code>).
...AvifQuality	50	Qualité AVIF (0-100). ≈ 45-55 ≈ WebP 80-85 visuellement (le gain réel dépend du contenu et de l'encodeur). L'alpha est préservé.

5. Architecture technique

Cette section décrit le fonctionnement interne de l'extension. Elle s'adresse aux administrateurs techniques et aux développeurs souhaitant comprendre ou étendre le code.

5.1 Flux de traitement d'un fichier

Lorsqu'un fichier est téléversé ou traité par le rattrapage, les étapes sont les suivantes :

1. **Vérification** : le type MIME est-il dans la liste autorisée ? La taille est-elle sous la limite ? Le fichier est-il accessible en écriture ?
2. **Optimisation de l'original** (si activée) : recompression sans perte (PNG/JPEG ; GIF via gifsicle, §4.6). Garde-fou essentiel — le résultat n'écrase l'original que s'il est strictement plus petit ; sinon l'original est conservé intact.
3. **Resynchronisation des métadonnées** (1.8.1) : si la 1^{re} passe a réellement réduit l'original, les métadonnées MediaWiki (`img_sha1`, `img_size`, dimensions) sont resynchronisées. Sans cela, la détection de doublons et l'invalidation du cache des miniatures resteraient pointées sur les octets d'avant optimisation.
4. **Génération du WebP** (et de l'AVIF si activé) dans le répertoire dédié.
5. **Enregistrement** : les tailles (originale, optimisée, WebP) sont consignées en base de données pour les statistiques.

Garde-fou anti-gonflage. L'optimisation écrit toujours dans un fichier temporaire et ne remplace l'original que si la nouvelle version est strictement plus petite. Cela garantit qu'une optimisation ne peut jamais, en aucun cas, augmenter la taille d'un fichier — un écueil classique lorsqu'un ré-encodage produit un fichier plus gros que la source (cas fréquent sur des PNG déjà optimisés par un autre outil). Les écritures sont **atomiques** (fichier temporaire unique + `rename`), si bien qu'un lecteur concurrent ne voit jamais un fichier à moitié écrit.

5.2 Diffusion via la balise picture

À l'affichage d'une page (action *view*), un hook réécrit le HTML des images pour les envelopper dans une balise `<picture>`. Celle-ci propose la source WebP (et l'AVIF avant elle si activé), avec un repli sur l'image d'origine. Le navigateur choisit automatiquement le meilleur format qu'il supporte. La gestion des écrans haute densité (Retina) est prise en compte : le `srcset` liste les différentes tailles disponibles (1x, 1,5x, 2x).

Point important. L'extension n'émet une source que si le fichier WebP/AVIF correspondant existe réellement sur le disque. C'est volontaire : contrairement à un `<img srcset>`, une source manquante dans une balise `<picture>` ne se replie pas automatiquement et afficherait une image cassée. Si le WebP d'une taille donnée n'est pas disponible, l'image d'origine est donc servie intacte, sans dégradation. **Corollaire (correctif 1.8.1)** : sous GD, un GIF **animé** ne reçoit pas de WebP/AVIF (GD ne décode que la 1^{re} frame) — le GIF animé d'origine est conservé pour ne jamais figer l'animation. Imagick et libvips produisent un WebP/AVIF animé et ne sont pas concernés.

5.3 Génération des WebP : à la création et à la demande

La couverture WebP des miniatures repose sur deux mécanismes complémentaires, ce qui garantit qu'aucune taille réellement utilisée ne reste sans WebP :

- **À la création** : lorsqu'une miniature est générée par MediaWiki, un hook produit immédiatement sa version WebP. C'est le cas pour tout nouveau téléversement et toute nouvelle taille demandée pour la première fois.
- **À la demande** : lors du rendu d'une page, si une image référence une miniature dont le WebP est absent mais dont la source (la miniature d'origine) existe sur le disque, l'extension génère le WebP manquant à ce moment-là, dans le même flux que celui qui sert déjà la page (borné par `OnDemandThumbLimit`).

Ce double mécanisme corrige une limite structurelle : le hook de création ne se déclenche que lorsqu'une miniature est physiquement générée, jamais lorsqu'une miniature déjà présente sur le disque est servie telle quelle. Sans la génération à la demande, les miniatures créées avant l'installation de l'extension, ou les tailles que MediaWiki ne régénère pas (par exemple des largeurs fixes d'infobox), ne recevraient jamais de WebP.

Performance. Le tout premier rendu d'une page « froide » peut être légèrement plus lent, car il génère à cet instant les WebP manquants des tailles qu'elle utilise. Les rendus suivants sont instantanés (le WebP est en cache sur le disque). Si une génération échoue de façon répétée (par exemple un problème de droits sur le répertoire WebP), surveillez les journaux et vérifiez les droits du répertoire de destination.

5.4 Traitement asynchrone

Le rattrapage des fichiers existants passe par la file de tâches (JobQueue) de MediaWiki, et non par les requêtes web. Deux types de tâches sont définis : l'optimisation d'image standard et la recompression zopflipng/oxipng. Ce découplage évite de ralentir les pages et permet d'étaler la charge dans le temps. Un troisième usage : le second passage zopflipng des **miniatures** nouvellement générées est lui aussi confié à un job (§8.7), pour ne jamais ralentir le rendu. En mode « gros wiki » (§4.8), on court-circuite la file au profit du traitement CLI (§8.6).

5.5 Stockage et cache

Les données de suivi sont conservées dans la table `vtmo_image_optimization` (une ligne par fichier traité). Les statistiques agrégées sont mises en cache via le cache d'objets (`WANObjectCache`) avec une courte durée de vie, pour éviter de recalculer des sommes coûteuses à chaque consultation. Le cache est invalidé automatiquement à chaque écriture. Les requêtes de statistiques sont portables sur MySQL/MariaDB, SQLite et PostgreSQL.

6. Pages spéciales

L'extension fournit trois pages spéciales, accessibles aux administrateurs (droit `vaulttecmediaoptimizer-admin`, accordé par défaut au groupe `sysop`). Elles disposent de noms canoniques anglais et d'alias français.

Rôle	Alias français	Nom canonique
État / diagnostic	Spécial:VaultTec_État	Special:VTMOStatus
Rattrapage	Spécial:VaultTec_Optimisation	Special:VTMOBackfill
Statistiques	Spécial:VaultTec_Statistiques	Special:VTMOStats

6.1 VaultTec_État (diagnostic)

Cette page vérifie l'ensemble de l'environnement et affiche chaque contrôle avec un code couleur : vert (conforme), orange (avertissement non bloquant), rouge (problème bloquant), ou information neutre. Elle couvre la version de PHP et de MediaWiki, la mémoire, les bibliothèques d'images et leur support WebP, le **moteur actif** (Imagick/GD/vips) et sa sonde, le système de fichiers et les répertoires, la table de base de données, l'état du second passage PNG (moteur sélectionné, exécution shell, binaire), la disponibilité de **gifsicle** et de l'**AVIF/AV1**, la compatibilité avec d'autres extensions, et la configuration. C'est le premier endroit à consulter en cas de comportement anormal.

6.2 VaultTec_Optimisation (rattrapage)

Cette page pilote le traitement des fichiers déjà présents. Elle indique combien de fichiers restent à traiter et permet de programmer des lots dans la file de tâches. Si le second passage PNG est disponible, un bouton dédié permet de lancer la recompression zopflipng/oxipng des fichiers existants. Le bouton n'apparaît que si le moteur est correctement détecté.

6.3 VaultTec_Statistiques

Cette page présente, sous forme de tableaux, une vue d'ensemble des gains réalisés par l'extension. Toutes les valeurs sont calculées à partir des tailles réellement mesurées sur le disque (avant / après chaque étape).

6.3.1 Les trois mesures de gain

Un fichier passe par jusqu'à trois états : la taille à laquelle il a été traité pour la première fois par l'extension, sa taille après le premier passage (optimisation à l'upload), et sa taille après le second passage (oxipng/zopflipng). La page affiche donc trois mesures complémentaires plutôt qu'un seul chiffre :

- **Espace total économisé** : de l'état initial au fichier réel actuel. C'est le chiffre le plus représentatif du gain global.
- **Espace économisé (1^{er} passage)** : ce qu'a gagné l'optimisation au moment de l'upload.
- **Espace économisé (2nd passage)** : ce qu'a gagné en plus la recompression oxipng/zopflipng (n'apparaît que si un second passage a eu lieu).

Gain faible si les fichiers étaient déjà optimisés. La taille « de départ » enregistrée est celle du fichier lorsque l'extension l'a traité pour la première fois, et non la photo brute d'origine. Si vos fichiers ont déjà été optimisés auparavant, il ne reste que peu à gagner, et le pourcentage « originaux » sera naturellement faible. Ce n'est pas une contre-performance : cela signifie simplement que vos fichiers étaient déjà propres. Le gain brut depuis l'image jamais optimisée ne peut pas être reconstitué, car cette taille n'est conservée nulle part.

6.3.2 Bilan de l'espace disque

Cette section répond à une question essentielle et souvent mal comprise : l'extension fait-elle gagner ou perdre de l'espace disque ? Elle additionne l'espace gagné en optimisant les originaux, puis en soustrait l'espace occupé par les fichiers WebP générés (un par image, en plus de l'original). Le résultat net est presque toujours négatif : les WebP pèsent davantage que ce qui est gratté sur les originaux.

Le WebP coûte du disque, mais économise de la bande passante. Il est normal et attendu que le bilan disque soit négatif. Le but du WebP n'est pas d'économiser du stockage serveur, mais de réduire le poids des images envoyées aux visiteurs (bande passante). L'extension échange donc un peu d'espace disque (relativement peu coûteux) contre une économie de bande passante (le vrai bénéfice). Voir le §16 pour un chiffrage mesuré et honnête.

6.3.3 Bande passante économisée par page (estimation)

Cette section illustre l'économie de bande passante sur une page-type. Une page de wiki affiche des miniatures (et non les originaux en pleine taille) : la simulation suppose un nombre fixe de miniatures par page, d'une taille de référence, et applique le taux de réduction WebP moyen réellement observé sur le wiki. Les hypothèses sont affichées explicitement.

Estimation, pas mesure. L'extension ne suit pas les consultations réelles. Le **pourcentage** de réduction est fiable (il vient des tailles réelles), mais le **volume absolu** dépend du nombre et de la taille des images réellement affichées, ainsi que des caches navigateur et CDN qui réduisent fortement le trafic effectif. Considérez ce chiffre comme un ordre de grandeur, non comme une facture (cf. §16).

6.3.4 Autres sections

La page présente également les taux de compression (originaux sans perte et versions WebP), une répartition par format (PNG, JPEG, GIF), une section dédiée au second passage PNG (espace récupéré, nombre d'originaux et de miniatures recompressés), une répartition par moteur de traitement, et la liste des éventuels fichiers en échec.

Lecture des statistiques pendant un traitement. Lorsqu'un rattrapage ou une réparation est en cours, les chiffres évoluent à chaque rafraîchissement et ne deviennent définitifs qu'une fois le traitement terminé (la ligne « Progression des originaux » atteint 100 %). Il est normal de voir certains pourcentages évoluer, voire être temporairement négatifs si des données antérieures sont encore en cours de correction.

7. Installation des binaires de second passage

Le second passage PNG (section 4.4) requiert l'installation d'un binaire externe : `zopflipng` ou `oxipng`. Cette section détaille leur mise en place, en tenant compte d'un piège fréquent sur les hébergements mutualisés.

7.1 Le piège `open_basedir`

De nombreux hébergements (notamment sous Plesk) restreignent les chemins que PHP peut lire et exécuter via la directive `open_basedir`. Typiquement, PHP n'a accès qu'au répertoire du site (par exemple `/var/www/vhosts/votre-site/`) et à `/tmp/`, mais pas à `/usr/bin/`. Conséquence : même si un binaire est installé dans `/usr/bin/` et fonctionne parfaitement en ligne de commande, PHP ne pourra ni le voir ni l'exécuter depuis le contexte web. C'est une cause d'erreur très courante et déroutante, car le binaire « existe » mais reste inutilisable par l'extension.

Règle d'or. Placez toujours le binaire de recompression à l'intérieur de l'arborescence du wiki (par exemple dans un sous-dossier `bin/`), de sorte qu'il soit couvert par `open_basedir`. Configurez ensuite le **chemin absolu complet** vers ce binaire, et non un simple nom de commande.

7.2 Installation d'`oxipng` (recommandé)

`oxipng` est distribué sous forme de binaire précompilé. Attention toutefois : les binaires génériques téléchargés depuis les dépôts publics peuvent exiger une version de la bibliothèque système (glibc) plus récente que celle de votre serveur, provoquant une erreur du type « GLIBC_2.34 not found ». La méthode la plus fiable est alors de compiler `oxipng` sur la machine elle-même (via l'outil `cargo` de Rust), puis de copier le binaire obtenu dans l'arborescence du wiki.

7.2.1 Mise en place du binaire

Une fois `oxipng` disponible, copiez-le dans un dossier `bin/` du wiki et donnez-lui les bons droits. L'utilisateur propriétaire doit correspondre à celui qui exécute PHP (souvent un compte dédié sous Plesk) :

```
mkdir -p /chemin/vers/wiki/httpdocs/bin
cp /source/oxipng /chemin/vers/wiki/httpdocs/bin/oxipng
chmod 755 /chemin/vers/wiki/httpdocs/bin/oxipng
# Ajuster propriétaire:groupe selon votre hébergement
chown utilisateur:groupe /chemin/vers/wiki/httpdocs/bin/oxipng
```

Vérifiez ensuite que le binaire répond, en tant qu'utilisateur du wiki :

```
/chemin/vers/wiki/httpdocs/bin/oxipng --version
```

7.2.2 Configuration

Renseignez le chemin absolu dans `LocalSettings.php` (et non le nom `oxipng` seul) :

```
$wgVaultTecMediaOptimizerZopfliEnabled = true;
$wgVaultTecMediaOptimizerPngEngine = 'oxipng';
$wgVaultTecMediaOptimizerOxipngBinary = '/chemin/vers/wiki/httpdocs/bin/oxipng';
```

Rechargez ensuite Spécial:VaultTec_État : la ligne « Binaire » de la section recompression PNG doit passer au vert avec le chemin affiché.

7.3 Vérifier l'exécution shell côté web

L'extension a besoin que PHP puisse lancer des processus externes (fonctions `proc_open` ou `exec`). Si ces fonctions figurent dans la directive `disable_functions` de PHP, le second passage (ainsi que `gifsicle` et `vips`) est impossible. La page VaultTec_État indique l'état de l'exécution shell dans le contexte web — c'est ce contexte qui compte, et il peut différer de la ligne de commande. En cas de blocage côté web seulement, retirez `proc_open` et `exec` de `disable_functions` dans les réglages PHP (par exemple via le panneau Plesk).

8. Maintenance

8.1 Lancer le rattrapage initial

Après l'installation, les fichiers déjà présents ne sont pas encore traités. Lancez le rattrapage depuis **Spécial:VaultTec_Optimisation**, qui programme les lots dans la file de tâches de MediaWiki.

Important — préférez la ligne de commande pour les gros volumes. La page spéciale ne traite pas les fichiers directement : elle planifie des tâches dans la file de jobs. Par défaut, MediaWiki exécute ces tâches à la fin des requêtes des visiteurs (paramètre `$wgJobRunRate`). Comme chaque tâche d'optimisation lance un outil externe qui consomme du temps processeur, la requête du visiteur attend la fin de la tâche avant de rendre la main : le wiki paraît alors ralenti, alors que ce sont en réalité vos visiteurs qui exécutent le traitement à votre place. Pour un rattrapage ou une recompression de masse, lancez plutôt le traitement en ligne de commande (SSH), dans un processus séparé qui n'impacte pas les visiteurs.

Deux approches propres pour un gros volume : (1) le plus simple, utiliser les scripts de maintenance en SSH dans un `screen` (ils tournent dans un processus isolé) ; (2) si vous tenez à passer par la file, désactivez son exécution sur les requêtes web le temps du traitement avec `$wgJobRunRate = 0;`, puis videz la file vous-même en SSH avec `runJobs.php`.

```
screen -S vtmo-jobs
php maintenance/run.php runJobs.php --type VaultTecMediaOptimizerOptimizeImage
```

Le réglage `$wgJobRunRate`. Augmenter `$wgJobRunRate` (comme le suggèrent certains messages génériques de MediaWiki) **aggrave** le ralentissement pour les visiteurs au lieu de l'atténuer, puisque davantage de tâches lourdes s'exécutent sur leurs requêtes. Pour des traitements lourds, la bonne valeur est **0** (aucune tâche sur les requêtes web), combinée à un `runJobs.php` lancé en SSH, ou tout simplement les scripts de maintenance dédiés (§8.6).

8.2 Réparer des originaux indûment gonflés

Un script de maintenance dédié, `repairBloatedOriginals.php`, permet de réparer des fichiers dont la version « optimisée » serait, à la suite d'un défaut historique, plus grosse que l'original. Il recomprime chaque fichier concerné avec le moteur PNG configuré (oxipng recommandé pour la rapidité), corrige les tailles enregistrées en base afin que les statistiques redeviennent cohérentes, et resynchronise les métadonnées de MediaWiki. Commencez toujours par un essai à vide (`--dry-run`), qui liste les fichiers concernés sans rien modifier :

```
php maintenance/run.php .../repairBloatedOriginals.php --dry-run
php maintenance/run.php .../repairBloatedOriginals.php --max=20 # valider le rythme
php maintenance/run.php .../repairBloatedOriginals.php             # tout (dans un screen)
```

Astuce — chemin du script. Si l'invocation par chemin relatif échoue (« Script not found »), utilisez le **chemin absolu** vers le script. Lancez toujours le script via `maintenance/run.php`, et non en l'exécutant directement, conformément aux recommandations de MediaWiki.

Le script est relançable sans risque : il ne reprend que les fichiers encore concernés et ignore ceux déjà réparés. Pour un gros volume, lancez-le dans un `screen` ou `tmux` afin qu'il survive à une déconnexion.

8.3 Recompresser avec un second moteur (oxipng puis zopflipng)

Il est possible d'obtenir la meilleure compression PNG possible en traitant d'abord tous les fichiers avec oxipng (rapide), puis en repassant avec zopflipng (plus lent, mais DEFLATE plus dense) pour grappiller les derniers pourcents. Le script `recompressOriginals.php` est prévu pour cela.

À comprendre avant tout. Le PNG étant un format sans perte, chaque moteur ré-encode l'image à partir des pixels décodés. Enchaîner oxipng puis zopflipng ne cumule donc **pas** les gains des deux : le résultat final est déterminé par le dernier moteur qui écrit. Concrètement, « oxipng puis zopflipng » donne le même résultat que « zopflipng seul ». L'intérêt de l'enchaînement n'est pas d'additionner les gains, mais de dégonfler vite le gros du stock avec oxipng, puis de ne lancer le moteur lent que pour la finition. Une protection intégrée ne conserve la sortie du second moteur que si elle est réellement plus petite : un fichier n'est jamais agrandi.

8.3.1 Premier puis second passage

Avec le moteur configuré (par exemple oxipng), traitez tous les PNG qui n'ont pas encore subi le second passage. Utilisez le **chemin absolu** du script (et de `run.php`) pour éviter l'erreur « Script not found » :

```
# 1er passage (moteur configuré, ex. oxipng)
/opt/plesk/php/8.4/bin/php /abs/wiki/maintenance/run.php \
    /abs/wiki/extensions/VaultTecMediaOptimizer/maintenance/recompressOriginals.php
# 2e passage : passer PngEngine = 'zopflipng' dans LocalSettings, puis :
... recompressOriginals.php --reset
```

L'option `--reset` remet les PNG déjà traités à l'état « à faire » pour que le nouveau moteur les reprenne. `--dry-run` prévisualise, `--max=N` limite le nombre de fichiers. Comme zopflipng est lent, lancez ce second passage dans un `screen`.

8.3.2 Cas des miniatures

Ce script ne traite que les originaux. Les miniatures sont recompressées automatiquement par MediaWiki au moment où elles sont (re)générées, via un hook. Pour qu'un nouveau moteur s'applique aux miniatures existantes, il faut donc les purger pour forcer leur régénération. Il n'existe volontairement pas de re-parcours en masse des miniatures, car l'énumération fiable des miniatures sur disque est fragile et dépend de la configuration.

8.4 Purge du cache après modification des messages

Si vous modifiez les fichiers de traduction (i18n), pensez à reconstruire le cache de localisation :

```
php maintenance/run.php rebuildLocalisationCache --force
```

8.5 En-têtes de cache HTTP (recommandation)

Pour tirer pleinement parti des WebP générés, configurez votre serveur web afin que les fichiers WebP (et AVIF) soient mis en cache longuement par les navigateurs — par exemple, sous Apache, en appliquant un en-tête `Cache-Control` à durée longue sur les fichiers `.webp` du répertoire `images_webp/`. C'est un levier important et souvent négligé pour réduire la bande passante (voir §16).

8.6 Backfill en ligne de commande — `optimizeImages.php` NOUVEAU

Pour les gros wikis, un script dédié réalise le rattrapage **directement en ligne de commande** (1^{re} passe + WebP) **sans passer par la file de tâches** — idéal pour ne pas impacter les visiteurs. Il honore le moteur configuré (vips inclus). Combinez-le avec le mode gros wiki (§4.8) :

```
// LocalSettings.php (mode gros wiki)
$wgVaultTecMediaOptimizerUseJobQueue = false;

screen -S vtmo
php maintenance/run.php ../optimizeImages.php --dry-run           # aperçu sans rien modifier
php maintenance/run.php ../optimizeImages.php --batch=200        # traiter par lots
php maintenance/run.php ../optimizeImages.php --format=gif       # restreindre à un format
php maintenance/run.php ../optimizeImages.php --start=Foo.png --max=500 --force
```

Options : `--dry-run`, `--batch`, `--max`, `--start`, `--force`, `--format` (sous-ensemble des formats configurés, intersecté avec `...Formats`), `--purge-queue`. Le script `purgeQueue.php` vide la file Optimisation (et, avec `--include-zopfli`, la file du second passage) ; `--dry-run` compte sans supprimer.

Une seule voie à la fois pour un même corpus : soit le rattrapage par la file (`runJobs.php`), soit le CLI (`optimizeImages.php`). Ne lancez pas les deux en même temps sur le même stock : les écritures sont atomiques (pas de corruption), mais vous feriez du travail en double. Utilisez `--purge-queue` au besoin.

8.7 Le second passage des miniatures : un job, à traiter hors-ligne NOUVEAU

Quand MediaWiki génère une nouvelle miniature PNG, l'extension produit immédiatement son WebP (rapide, < 50 ms). En revanche, le **second passage zopfli** sur cette miniature — purement un gain de disque — coûte **plusieurs secondes par fichier** (≈ 14 s, cf. §16). Le lancer pendant le rendu ferait payer ce temps au **visiteur** : une galerie de 30 PNG en cache froid pourrait ajouter des **minutes** à l'affichage d'une page. C'est pourquoi ce second passage n'est **pas** exécuté en ligne : il est confié à un **job** (`VaultTecMediaOptimizerThumbnailRecompress`) qui recomprime la miniature stockée en arrière-plan.

Pourquoi la génération via la ligne de commande est meilleure. Un job n'est utile que s'il est traité **hors des requêtes des visiteurs**. Par défaut, MediaWiki exécute la file en fin de requête (`$wgJobRunRate`) : le traitement lourd retombe alors sur un visiteur. La bonne configuration est `$wgJobRunRate = 0` ; + un `runJobs.php` lancé en **SSH/cron** (ou le backfill CLI `optimizeImages.php`, §8.6). Ainsi tout l'encodage lourd s'exécute sur un processus serveur dédié, **jamais sur le temps de réponse d'un visiteur** — c'est la même règle que pour l'optimisation des originaux (§8.1). En **mode gros wiki** (`UseJobQueue = false`, §4.8), ce job n'est pas enfilé du tout : l'administrateur recomprime les miniatures à son rythme.

9. Dépannage

9.1 « is not a valid JSON file » au démarrage

Cause : le fichier `extension.json` a été corrompu, le plus souvent par un transfert FTP en mode ASCII. **Solution** : re-transférez le fichier en mode binaire, ou décompressez l'archive directement sur le serveur. Vous pouvez vérifier la validité côté serveur et comparer la taille du fichier à celle de l'archive d'origine.

9.2 La page de statistiques affiche une erreur de colonne inconnue

Cause : la migration de base de données n'a pas été appliquée (colonne ou table manquante). **Solution** : exécutez `update.php` (section 3.3). Si une autre extension bloque l'updater, appliquez la migration manuellement (section 3.3.1).

9.3 Le binaire de recompression est introuvable (alors qu'il existe)

Cause la plus fréquente : `open_basedir` empêche PHP d'accéder au binaire situé hors de l'arborescence autorisée (typiquement dans `/usr/bin/`). Solution : copiez le binaire dans un dossier du wiki et configurez son chemin absolu (section 7). La page VaultTec_État affiche un message spécifique lorsqu'elle détecte ce cas. Autre cause : la mauvaise variable de configuration a été renseignée (avec `oxipng`, c'est `...0xipngBinary`, et non la variable `zopfli`) ; vérifiez aussi la casse exacte du nom. Erreur « GLIBC not found » : le binaire précompilé exige une glibc plus récente que celle du serveur — compilez-le sur la machine.

9.4 La recompression est très lente

Cause : le moteur `zopflipng` est intrinsèquement lent, surtout avec un nombre d'itérations élevé sur de gros fichiers. Solution : passez au moteur `oxipng` (`...PngEngine = 'oxipng'`), nettement plus rapide. À défaut, réduisez le nombre d'itérations `zopflipng` (par exemple à 5). Le résultat final en taille est très proche, pour un temps de traitement bien moindre (voir §16).

9.5 Les statistiques semblent incohérentes pendant un traitement

C'est normal : pendant un rattrapage ou une réparation, les valeurs évoluent à chaque rafraîchissement. Attendez la fin du traitement (« Progression » à 100 %) avant d'interpréter les chiffres. Par ailleurs, le total WebP affiché peut fluctuer temporairement, car la resynchronisation des métadonnées invalide certaines miniatures, qui sont ensuite régénérées.

9.6 Une image reste servie en PNG/JPEG plutôt qu'en WebP

Rechargez la page une fois (le premier rendu génère les WebP manquants), puis rechargez de nouveau. Vérifiez les droits d'écriture du répertoire WebP et que `OnDemandThumbLimit > 0`. Pour un **GIF animé sous GD**, l'absence de WebP est **voulue** (§5.2) afin de ne pas figer l'animation.

9.7 Droits sur les fichiers

Si vous lancez un script de maintenance, faites-le sous l'utilisateur qui exécute PHP (souvent le même que le serveur web), et non en tant que `root`. Lancer un traitement en `root` crée des fichiers appartenant à `root`, que PHP ne pourra plus écraser ensuite — source de conflits de droits lors des téléversements suivants.

10. Référence rapide

10.1 Tables de base de données

Table	Rôle
vtmo_image_optimization	Une ligne par fichier traité : tailles originale/optimisée/WebP, statut, etc.
vtmo_zopfli_thumb_stats	Statistiques agrégées des gains de recompression sur les miniatures.

10.2 Droit d'accès

Les trois pages spéciales requièrent le droit vaulttecmediaoptimizer-admin, accordé par défaut au groupe sysop (administrateurs).

10.3 Scripts de maintenance

Script	Rôle et options
optimizeImages.php NOUVEAU	Backfill CLI (1 ^e passe + WebP) sans la file ; --dry-run, --batch, --max, --start, --force, --format, --purge-queue.
purgeQueue.php NOUVEAU	Vide la file Optimisation (--include-zopfli pour le 2 ^e passage) ; --dry-run.
recompressOriginals.php	Recomprime les PNG originaux avec le moteur configuré ; --reset pour repasser un autre moteur, --dry-run, --max, --batch.
repairBloatedOriginals.php	Répare les originaux gonflés ; --dry-run, --max, --batch.

11. Foire aux questions

L'extension modifie-t-elle la qualité de mes images ?

Non pour les originaux : l'optimisation PNG est strictement sans perte (les pixels sont identiques), et les WebP de PNG sont générés en mode sans perte par défaut. Les JPEG sont ré-encodés à haute qualité (85 par défaut), ce qui est visuellement indistinguable de l'original dans l'immense majorité des cas. Vous pouvez ajuster la qualité WebP via `...WebPQuality`.

Que voient les visiteurs dont le navigateur ne supporte pas le WebP ?

Ils reçoivent l'image d'origine, intacte. La balise `<picture>` propose le WebP en premier mais conserve toujours la balise `<img>` d'origine en repli. Tous les navigateurs récents supportent le WebP ; le repli ne concerne que des navigateurs très anciens.

Dois-je activer le second passage (zopflipng/oxipng) ?

Ce n'est pas obligatoire. Le bénéfice principal pour les visiteurs (les WebP allégés) fonctionne sans lui. Le second passage n'apporte qu'un gain d'espace disque supplémentaire sur les PNG. Activez-le si le stockage serveur est une préoccupation ; sinon, laissez-le désactivé.

Quelle différence entre zopflipng et oxipng ?

Les deux recompressent les PNG sans perte. zopflipng compresse un peu plus mais est très lent ; oxipng est beaucoup plus rapide pour un résultat à quelques pourcents près. Pour du traitement de masse, oxipng est recommandé.

Puis-je changer de moteur après coup ?

Oui. Modifiez `...PngEngine` à tout moment. Les fichiers déjà traités ne sont pas re-traités automatiquement, mais tout nouveau traitement (ou une réparation) utilisera le nouveau moteur.

Les images GIF animées sont-elles préservées ?

Oui. L'optimisation `gifsicle` (§4.6) est strictement sans perte et sûre pour l'animation ; et sous GD, les GIF animés ne reçoivent jamais de WebP/AVIF figé (§5.2). Sans `gifsicle`, l'original du GIF n'est pas touché du tout.

Combien d'espace vais-je gagner ?

Cela dépend entièrement de votre médiathèque. Côté visiteurs, le WebP réduit souvent le poids des images servies de 25 à 50 % (voir §16 pour un chiffrage honnête au niveau des miniatures). Le gain d'espace disque sur les originaux est plus modeste et dépend de l'état de compression initial.

L'extension fonctionne-t-elle avec un stockage cloud (S3, Swift) ?

L'extension est conçue pour un système de fichiers local (`FSFileBackend`). Elle détecte les backends non locaux et le signale sur la page d'état. Un fonctionnement sur stockage objet n'est pas pris en charge en l'état.

Une image apparaît cassée après traitement. Que faire ?

C'est très rare grâce au garde-fou anti-gonflage et au repli `<img>`. Si cela arrive, vérifiez d'abord en accédant directement à l'URL de la miniature (elle doit renvoyer l'image avec un type `image/png` ou équivalent). Consultez ensuite les journaux et la section 9. Le fichier d'origine n'étant jamais supprimé, aucune donnée n'est perdue.

12. Désinstallation

L'extension est conçue pour être retirée proprement, sans laisser le wiki dans un état dégradé. Suivez ces étapes :

1. **Désactiver l'extension** : retirez (ou commentez) la ligne `wfLoadExtension( 'VaultTecMediaOptimizer' );` dans `LocalSettings.php`, ainsi que tous les réglages `$wgVaultTecMediaOptimizer*`.
2. **Purger le cache des pages** : les pages cesseront alors d'afficher les balises `<picture>` et reviendront aux balises `<img>` standards pointant vers les originaux. Les originaux étant intacts, les images s'affichent normalement.
3. **(Optionnel) Supprimer les fichiers WebP** : le répertoire `images_webp/` (et `images_avif/`) peut être supprimé pour récupérer l'espace. Il n'est plus utilisé une fois l'extension désactivée.
4. **(Optionnel) Supprimer les tables** : si vous ne comptez pas réinstaller l'extension, les tables `vtmo_image_optimization` et `vtmo_zopfli_thumb_stats` peuvent être supprimées. Retirez aussi le dossier de l'extension et les binaires installés.

Important. Les images d'origine ne sont jamais supprimées par l'extension. Après désinstallation, votre médiathèque reste complète et fonctionnelle. Seules les versions WebP/AVIF (régénérables) disparaissent.

```
DROP TABLE IF EXISTS vtmo_zopfli_thumb_stats;  
DROP TABLE IF EXISTS vtmo_image_optimization;
```

13. Sécurité

Cette section décrit les considérations de sécurité prises en compte par l'extension.

13.1 Exécution de binaires externes

Les binaires externes (gifsicle, zopflipng, oxipng, vips) sont appelés via les fonctions PHP `proc_open` ou `exec`. Les arguments sont transmis sous forme de **tableau** (et non d'une chaîne shell concaténée), ce qui évite toute interprétation par le shell et donc tout risque d'injection de commande. Les chemins de binaires proviennent exclusivement de la configuration de l'administrateur, jamais d'une entrée utilisateur.

13.2 Protection contre la traversée de répertoires

La génération de WebP à la demande lit le fichier source d'une miniature et écrit le WebP correspondant. Pour empêcher qu'un nom de fichier malveillant ne fasse sortir ces opérations de l'arborescence des médias, l'extension rejette tout nom contenant une séquence de traversée (`..`), un séparateur de chemin (`/` ou `\`) ou un octet nul. Les noms de fichiers MediaWiki légitimes ne contiennent jamais ces caractères.

13.3 Restriction aux types autorisés

Le traitement (optimisation, WebP) est strictement limité aux types MIME configurés (`image/png`, `image/jpeg`, `image/gif` par défaut). Tout autre type de fichier est ignoré. La taille maximale traitée est également plafonnée par `...MaxFileSize`.

13.4 Droits d'accès

Les trois pages spéciales (état, rattrapage, statistiques) sont réservées aux utilisateurs disposant du droit `vaulttecmediaoptimizer-admin`, accordé par défaut au seul groupe `sysop`. Les actions sensibles (lancement de rattrapage, recompression) ne sont donc pas accessibles aux utilisateurs ordinaires.

13.5 Innocuité des balises générées

La réécriture HTML se contente d'envelopper les balises `<img>` existantes dans une balise `<picture>` et d'ajouter une source WebP/AVIF. Les attributs d'origine de l'image sont préservés tels quels, et les URL sont systématiquement encodées. Aucune donnée arbitraire n'est injectée dans le HTML.

14. Glossaire

Terme	Définition
WebP	Format d'image moderne offrant une compression supérieure au PNG et au JPEG, avec ou sans perte. Supporté par tous les navigateurs récents.
AVIF	Format encore plus compact sur les photos à qualité égale (codec AV1), mais encodage plus lent et support encodeur variable. Expérimental ici (§4.9).
Sans perte (lossless)	Compression qui préserve exactement les données d'origine : l'image décompressée est strictement identique. C'est le cas de l'optimisation PNG et GIF/gifsicle de l'extension.
Avec perte (lossy)	Compression qui supprime des informations pour réduire la taille, au prix d'une légère altération. Utilisée pour les JPEG et, optionnellement, les WebP de JPEG.
Miniature (thumbnail)	Version redimensionnée d'une image, générée par MediaWiki pour l'affichage dans les pages (c'est ce qui est réellement servi).
Balise picture	Élément HTML permettant de proposer plusieurs sources d'image au navigateur, qui choisit la plus adaptée. Utilisée ici pour servir le WebP/AVIF avec repli sur l'original.
Backfill (rattrapage)	Traitement des fichiers déjà présents sur le wiki au moment de l'installation, par opposition aux nouveaux téléversements traités automatiquement.
Hook	Point d'extension de MediaWiki permettant à une extension de réagir à un événement (téléversement, génération de miniature, rendu de page, etc.).
File de tâches (JobQueue)	Mécanisme de MediaWiki exécutant des traitements en arrière-plan, hors des requêtes web, pour ne pas ralentir l'affichage des pages.
libvips	Bibliothèque/CLI de traitement d'image rapide et économe en mémoire (moteur optionnel, §4.7).
gifsicle	Outil externe d'optimisation GIF sans perte, sûr pour l'animation (§4.6).
open_basedir	Directive de PHP restreignant les répertoires accessibles. Source fréquente de blocage pour l'exécution de binaires situés hors de l'arborescence du site.
zopflipng / oxipng	Outils externes de recompression PNG sans perte. zopflipng privilégie la compression maximale, oxipng la rapidité.

15. Tutoriel pas-à-pas (de zéro à la production)

Ce tutoriel reprend, dans l'ordre, toutes les étapes pour installer l'extension, la configurer, traiter une médiathèque existante, et vérifier le résultat. Il est volontairement détaillé. Les chemins d'exemple (`/var/www/.../httpdocs`) sont à adapter à votre installation.

Avant de commencer. Prévoyez un accès **SSH** à votre serveur et un accès administrateur au wiki. Si vous n'avez qu'un accès FTP, certaines étapes (lancer les scripts, installer un binaire) ne seront pas possibles : un accès shell est nécessaire pour le traitement de masse.

Étape 1 — Installer les fichiers de l'extension

Décompressez l'archive dans le dossier `extensions/` de votre wiki. Le dossier doit s'appeler exactement `VaultTecMediaOptimizer`. En SSH :

```
cd /var/www/vhosts/votre-wiki/httpdocs/extensions
unzip /chemin/vers/VaultTecMediaOptimizer.zip
ls VaultTecMediaOptimizer/extension.json # doit exister
```

Si vous passez par FTP : transférez impérativement en **mode binaire** (un transfert ASCII corrompt `extension.json` et provoque « is not a valid JSON file »).

Étape 2 — Activer l'extension

```
wfLoadExtension( 'VaultTecMediaOptimizer' );
```

Rechargez une page du wiki. Si une erreur apparaît, vérifiez d'abord l'intégrité de `extension.json`.

Étape 3 — Créer les tables en base

```
php maintenance/run.php update --quick
```

S'il échoue à cause d'une autre extension, reportez-vous à la section 3.3.1 pour appliquer la migration manuellement.

Étape 4 — Vérifier l'état avec la page de diagnostic

Rendez-vous sur **Special:VTMOSstatus** (ou son alias Spécial:VaultTec_État). Les lignes sur PHP, MediaWiki, la bibliothèque d'image et le système de fichiers doivent être vertes. La section du second passage PNG peut être ambre (non configurée) : c'est normal et non bloquant. Ne passez à la suite que si les vérifications essentielles sont au vert.

Étape 5 — Comprendre ce qui sera optimisé

Type	Traitement
PNG	Optimisé sans perte (pixels identiques) à l'upload, puis recompressible plus fort via oxipng/zopflipng. WebP généré.
JPEG	Optimisé à l'upload (ré-encodage Imagick, qualité préservée — pas strictement sans perte). WebP généré.
GIF	Optimisé sans perte via <code>gifsicle</code> si activé (§4.6), animation préservée ; sinon original non modifié. WebP généré (animé sous Imagick/libvips).

Le cas JPEG, en toute transparence. L'optimisation JPEG des originaux passe par Imagick, qui ré-encode l'image : l'étape sur les tables de Huffman est sans perte, mais le ré-encodage en lui-même ne l'est pas à 100 %. L'extension préserve la qualité d'origine pour que la perte soit invisible, et ne conserve le résultat que s'il est plus petit. Si vous tenez à une optimisation JPEG strictement sans perte, retirez `image/jpeg` de `...Formats` (les WebP des JPEG continueront d'être générés) et utilisez un outil dédié comme `jpegtran` en dehors de l'extension. Dans tous les cas, les visiteurs reçoivent les WebP, non concernés par cette nuance.

Étape 6 — Lancer le rattrapage des fichiers existants

Les nouveaux téléversements sont traités automatiquement, mais les fichiers déjà présents doivent être rattrapés. Allez sur **Special:VTMOBackfill** (file) ou, recommandé pour les gros wikis, utilisez le CLI (§8.6). Assurez-vous que la file est traitée régulièrement, par exemple dans un `screen` :

```
screen -S vtmo-jobs
php maintenance/run.php runJobs.php --type VaultTecMediaOptimizerOptimizeImage
```

Suivez l'avancement sur **Special:VTMOStats**.

Étape 7 (optionnelle) — Installer un moteur de recompression PNG

Pour gagner davantage d'espace disque sur les PNG, installez oxipng (rapide) ou zopflipng (plus lent). Cette étape est facultative et ne profite qu'au stockage. Reportez-vous à la section 7, en particulier le piège `open_basedir`. C'est aussi le moment d'activer **gifsicle** (§4.6) et/ou **libvips** (§4.7) si souhaité.

Étape 8 (optionnelle) — Recompresser les PNG existants

```
screen -S vtmo-png
/opt/plesk/php/8.4/bin/php /abs/wiki/maintenance/run.php \
/abs/wiki/extensions/VaultTecMediaOptimizer/maintenance/recompressOriginals.php
```

Piège fréquent — « **Script not found** ». N'utilisez pas un chemin relatif : selon le répertoire courant, MediaWiki concatène mal les chemins et cherche le script dans `maintenance/extensions/...`. Donnez toujours le **chemin absolu** du script et de `run.php`.

Pour viser la meilleure compression, basculez ensuite sur `zopflipng` et relancez avec `--reset` (§8.3).

Étape 9 — Vérifier que les WebP sont bien servis aux visiteurs

Ouvrez une page du wiki contenant des images, puis inspectez le code (outils de développement). Une image optimisée doit être enveloppée dans une `<picture>` avec une source WebP :

```
<picture>
  <source srcset="/images_webp/...webp" type="image/webp">
  
</picture>
```

Si une image reste servie en PNG/JPEG, rechargez la page une fois (le premier rendu génère les WebP manquants), puis rechargez de nouveau.

Étape 10 — Purger les caches en fin de traitement

Une fois le rattrapage et l'éventuelle recompression terminés, purgez le cache des pages du wiki **et** du CDN, pour que les pages reflètent l'état final.

Ordre important. Faites la purge **après** la fin du traitement, pas pendant. Purger pendant la réparation est contre-productif : les miniatures en cours d'invalidation seraient re-mises en cache dans leur ancienne version.

Récapitulatif express

1. Décompresser dans `extensions/` (FTP binaire).
2. `wfLoadExtension( 'VaultTecMediaOptimizer' );` dans `LocalSettings`.
3. `php maintenance/run.php update --quick`.
4. Vérifier **Special:VTMOSstatus** (tout au vert).
5. Lancer le rattrapage (file ou CLI `optimizeImages.php`).
6. (Option) Installer/activer `gifsicle`, `libvips`, `oxipng`.
7. (Option) `recompressOriginals.php`, puis finition `zopflipng` avec `--reset`.
8. Vérifier les balises `<picture>` sur quelques pages.
9. Purger les caches (wiki + CDN) en fin de traitement.

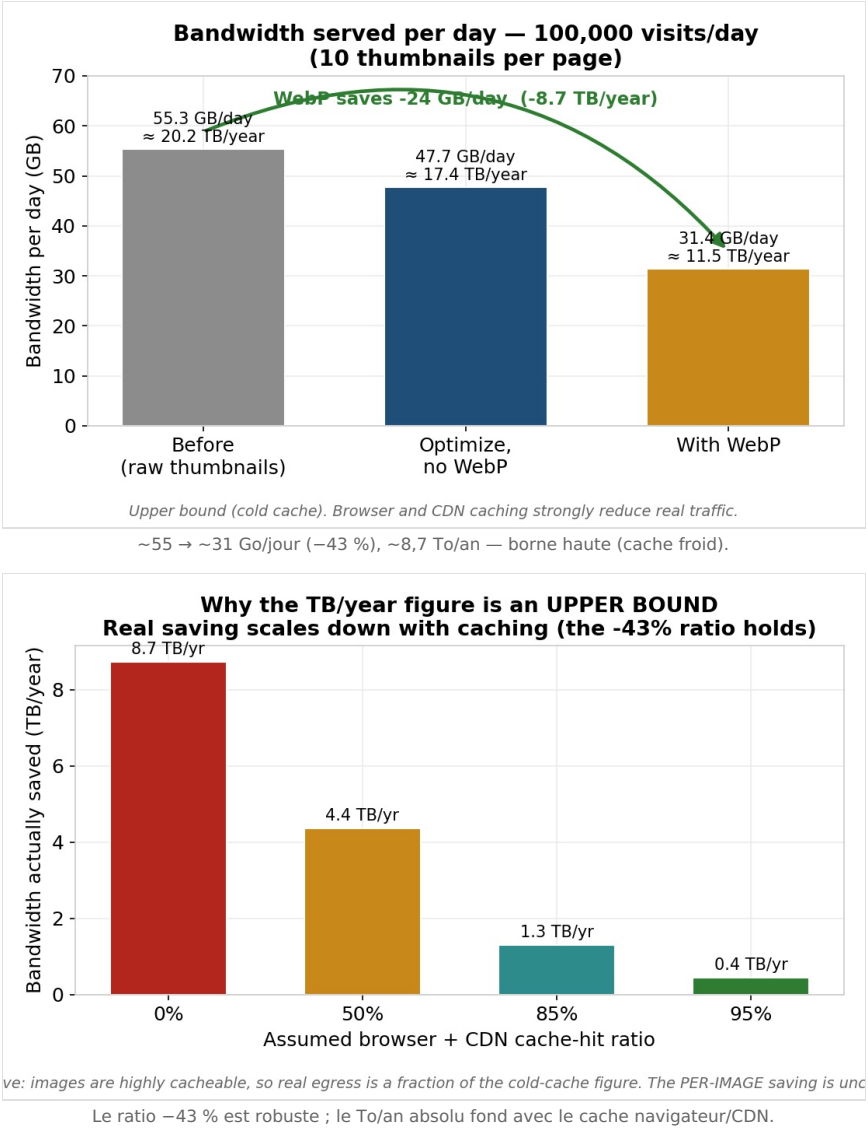
16. Benchmarks — coût / bénéfice NOUVEAU

Deux familles de chiffres. **(A)** un **modèle coût/bénéfice** du wiki réel, recalculé depuis des hypothèses explicites ; **(B)** des **mesures réelles** prises sur la machine de test (PHP 8.4, libvips 8.15.1, ImageMagick 6.9.12, gifsicle 1.94, zopflipng, PHP-GD ; **un seul cœur**). Tout est reproductible via `docs/benchmarks/model.py` et `tests/integration/pipeline.php`.

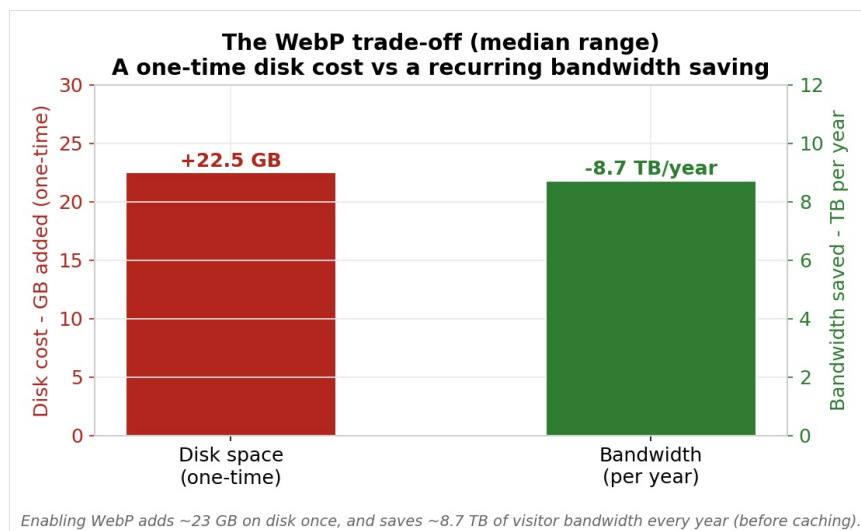
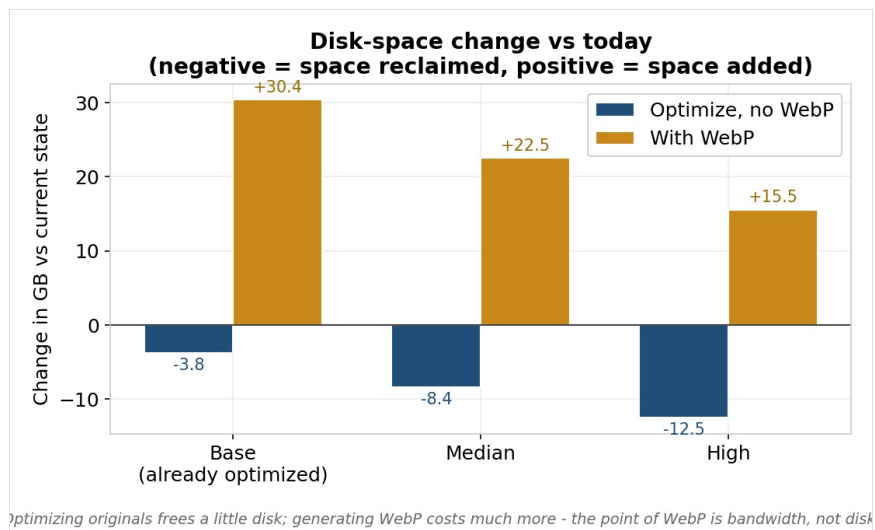
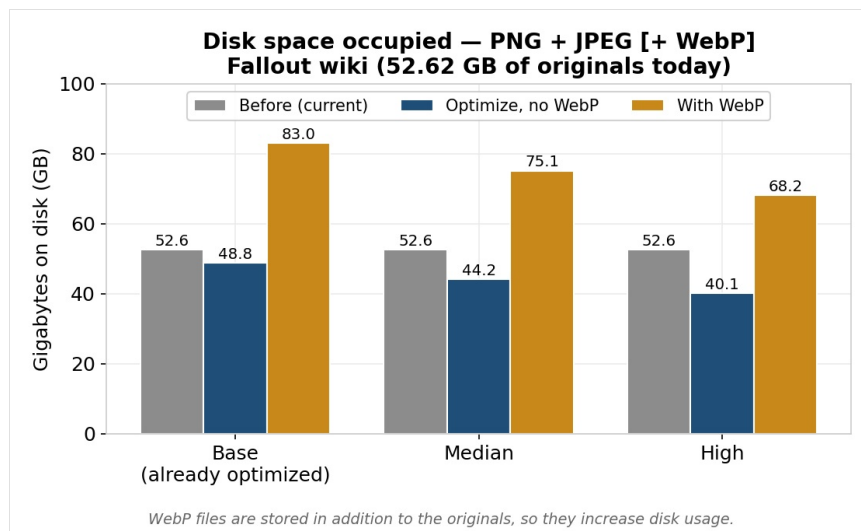
Hypothèses du modèle (unités décimales ; 365 j/an) : originaux **52,62 Go** de PNG+JPEG ; trafic en **borne haute (cache froid)** de **100 000 vues/jour × 10 miniatures** ; miniature servie moyenne 55,3 Ko ; optimisation sans perte de miniature –13,7 % ; WebP de miniature –34 % (≈ –43 % vs brut) ; recompression des originaux –7/–16/–24 % (bas/médian/haut) ; copies WebP ≈ 70 % de l'original optimisé, stockées **en plus**.

Cadrage honnête. Les pages servent des **miniatures** (déjà petites), pas les originaux, et le WebP **ajoute** du disque. On sépare donc la *bande passante servie* (le vrai bénéfice) de la *compression par fichier*.

16.1 Bande passante servie — le vrai bénéfice

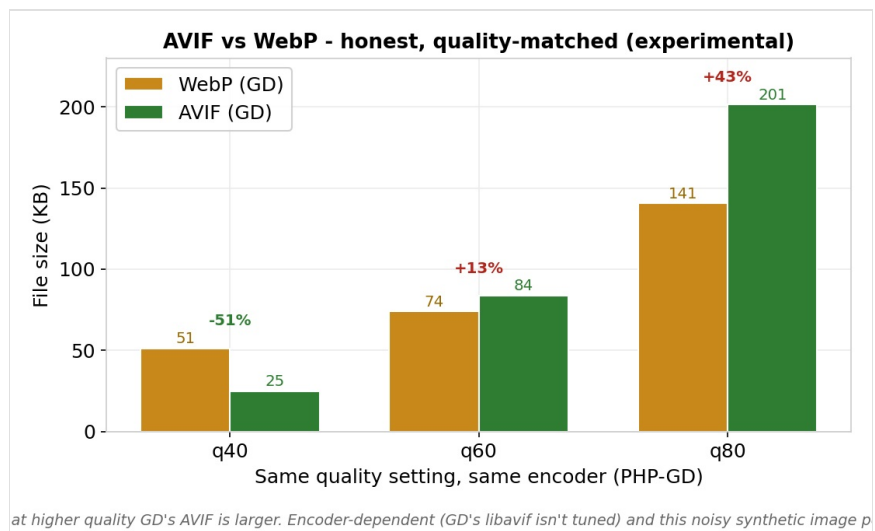
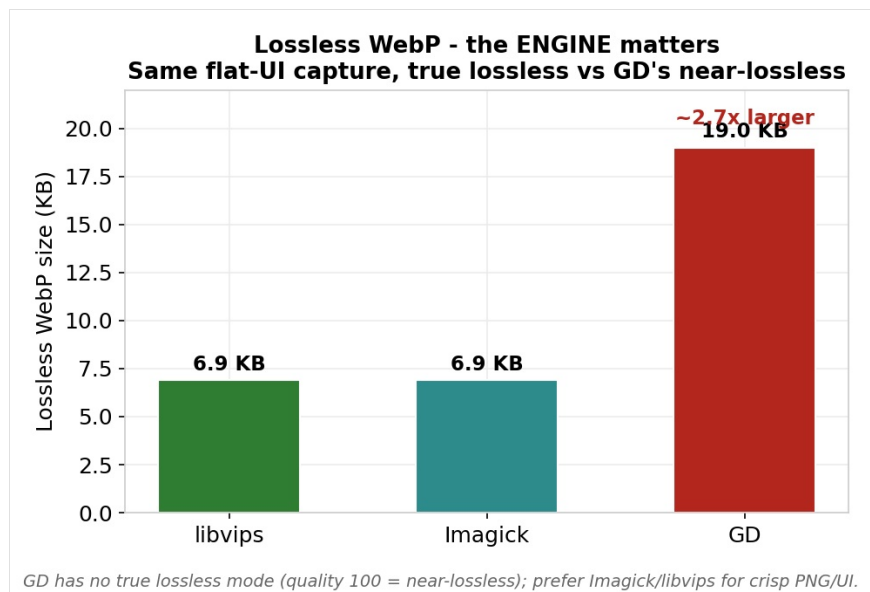
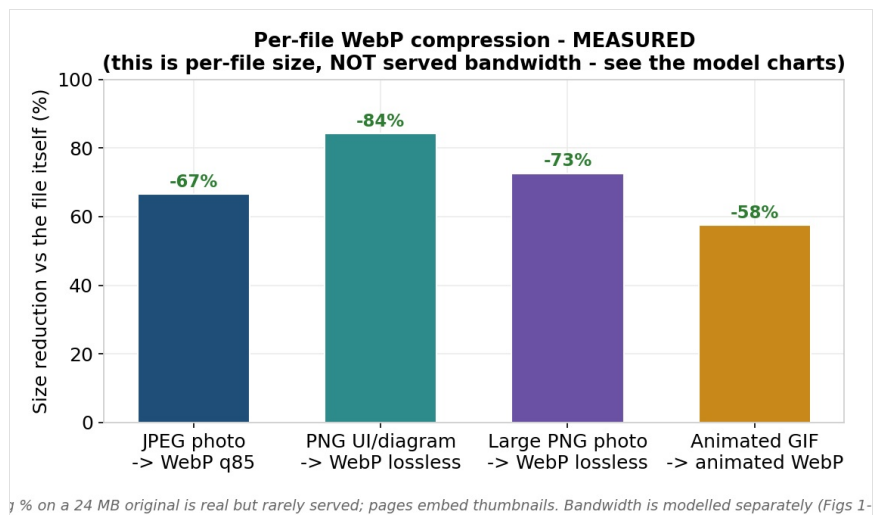


16.2 Disque : le WebP en ajoute



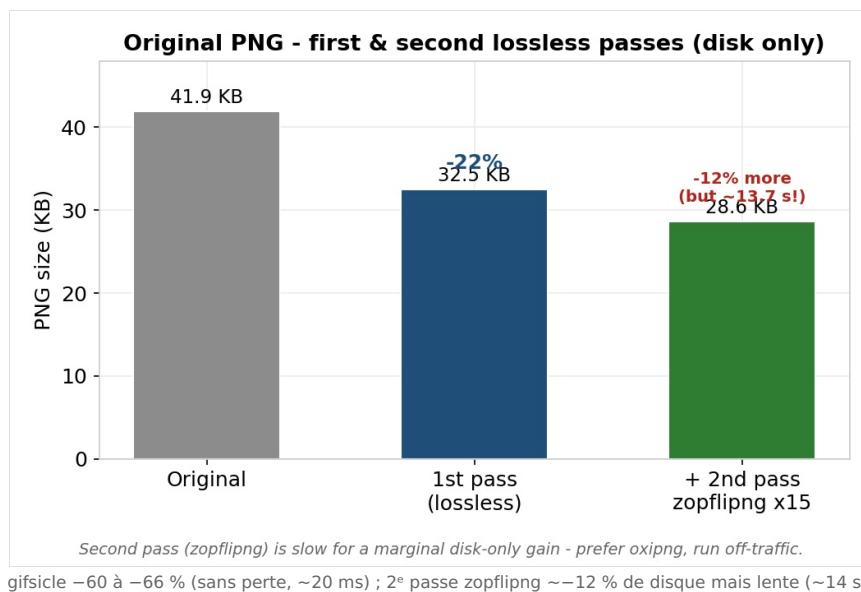
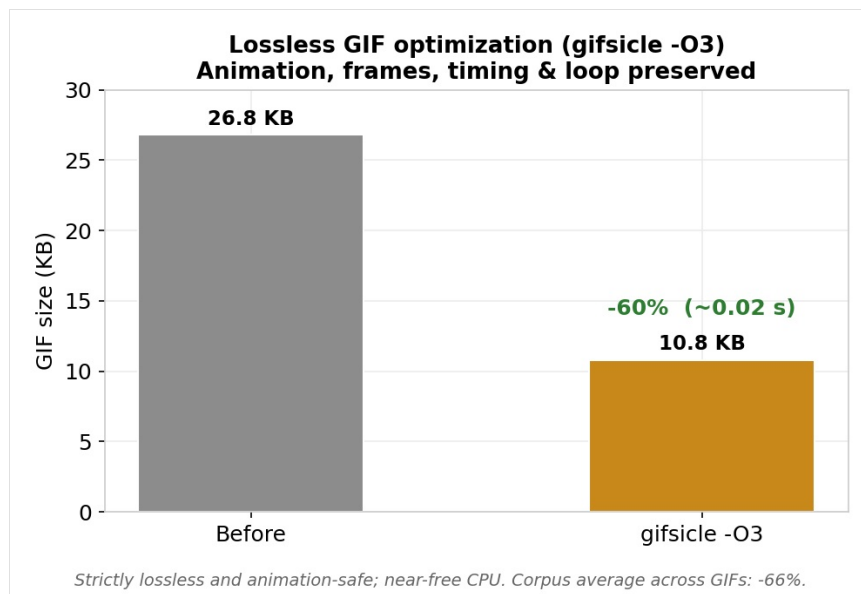
Le disque monte (52,6 → ~75 Go au médian) ; l'arbitrage : coût disque ponctuel (~+22 Go) vs économie de bande passante récurrente.

16.3 Compression par fichier (mesurée) — ≠ bande passante

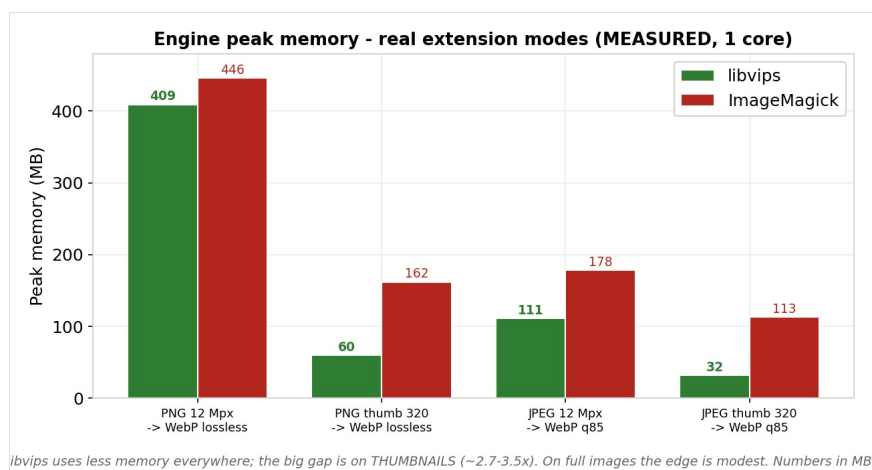


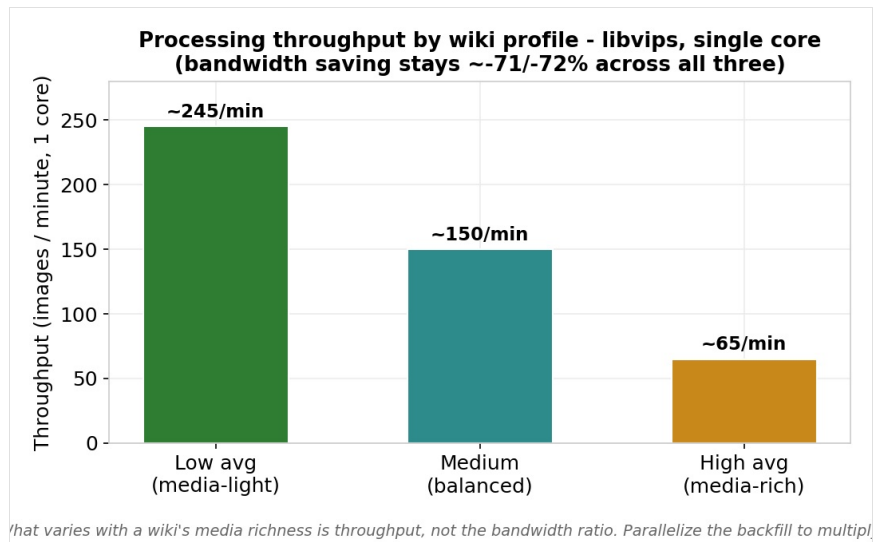
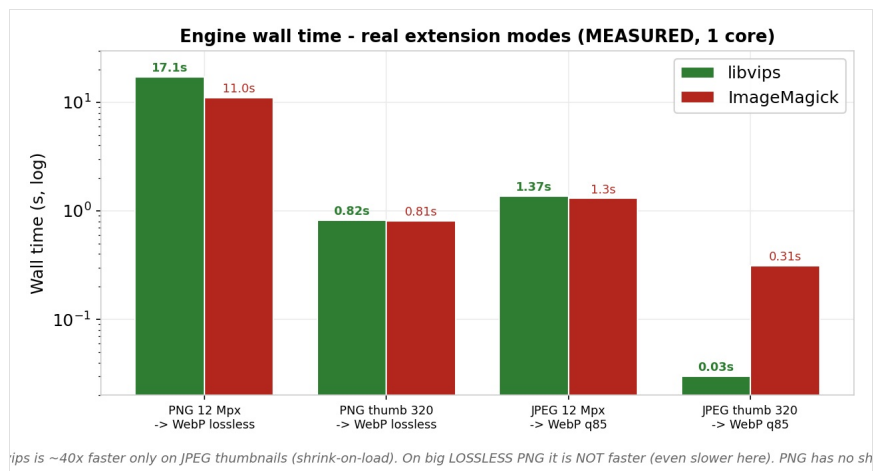
Gains par fichier (pas le trafic) ; le moteur compte (GD $\approx 2,7\times$ plus gros en lossless) ; à qualité égale, l'AVIF n'est pas systématiquement plus petit (dépend du contenu/encodeur).

16.4 GIF & 2^e passe PNG (disque)



16.5 Moteur : mémoire, vitesse, débit





Mesuré aux vrais réglages (PNG→WebP lossless, JPEG→q85) : libvips = moins de mémoire partout (surtout miniatures) ; ~40× plus vite uniquement sur les miniatures JPEG (shrink-on-load), PAS plus rapide sur gros PNG lossless. Débit ~245→~65 img/min selon la richesse média.

1. Overview

2. Prerequisites & dependencies — 2.5 installing the dependencies (new)

3. Installation

4. Configuration — 4.6 GIF/gifsicle, 4.7 libvips, 4.8 large-wiki mode, 4.9 AVIF (new)

5. Technical architecture · 6. Special pages · 7. Second-pass binaries

8. Maintenance — 8.6 CLI backfill (new) · 9. Troubleshooting

10. Quick reference · 11. FAQ · 12. Uninstallation · 13. Security · 14. Glossary

15. Step-by-step tutorial · 16. Benchmarks — cost / benefit (new)

What's new since v1.4.1 (through 1.8.1). This manual reproduces the full 1.4.1 documentation and integrates everything added since: optional **libvips** image engine (§4.7), **lossless GIF optimization** via gifsicle (§4.6), **large-wiki CLI mode** (§4.8 and §8.6), **experimental AVIF** (§4.9), **dependency installation** (§2.5), a measured, honest **cost/benefit benchmarks** section (§16), and the **1.8.1 fixes** (metadata refresh after the first pass; animated GIFs never frozen under GD). Experimental items are marked **EXPERIMENTAL**, additions **NEW**.

1. Overview

Vault-Tec Media Optimizer is a MediaWiki extension that reduces the weight of a wiki's media in two complementary ways: it optimizes the original files (PNG, JPEG, GIF) losslessly, and it generates a WebP version (and, optionally, AVIF) of each image, served automatically to browsers through a `<picture>` tag. Visitors receive substantially lighter images with no visible loss of quality, which speeds up page loads and reduces bandwidth.

The extension is designed to work transparently: newly uploaded files are processed automatically, and a backfill mechanism lets you process every file already present. Three special pages provide full diagnostics, backfill control, and detailed statistics.

1.1 How it works

- **The optimized original:** the source file (PNG/JPEG/GIF) is recompressed losslessly. The pixels stay strictly identical; only the file size on disk goes down. This saving is about server storage space.
- **The WebP version:** a WebP copy is generated and stored separately. This much lighter version is the one sent to compatible browsers. This saving is about bandwidth and load speed for visitors.

The `<picture>` tag inserted into the HTML lets the browser choose automatically: WebP (or AVIF) if it supports it (all modern browsers), the original format otherwise. The original image therefore always remains available, guaranteeing full compatibility and the correct operation of features such as the media viewer.

1.2 What the extension does not do

- It never degrades the visual quality of original images (strictly lossless optimization for PNG; high-quality re-encoding for JPEG).
- It does not delete original files: they stay in place and accessible.
- For **GIF**, optimizing the original is now possible and **strictly lossless** via `gifsicle` (\$4.6), animation-safe and **off by default**. Without `gifsicle`, the GIF original is left untouched (to take no risk with the animation) and only its WebP is produced — the historical behaviour.
- It depends on no external or cloud service: all processing happens on your server.

2. Prerequisites & dependencies

The extension requires an environment meeting the following conditions. The **VTMOStatus** special page (section 6.1) automatically checks all of these prerequisites and flags any problem.

2.1 Minimum versions

Component	Required version
MediaWiki	1.43 or higher
PHP	8.1 or higher
Image library	Imagick (recommended) or GD, with WebP support

2.2 Image processing library

The extension uses Imagick (the PHP interface to ImageMagick) by preference, as it is more faithful and complete. In its absence, it automatically falls back to GD, present by default in most PHP installations. In both cases, WebP support must be enabled in the library, otherwise WebP generation is impossible.

Tip. Imagick is strongly recommended. It handles PNG transparency and color profiles better and offers finer control over compression (and true lossless WebP, where GD only approximates). The VTMOStatus page clearly indicates which library is in use and whether WebP support is present. *See also the **libvips** engine (§4.7), a fast, memory-light alternative at scale.*

2.3 File system

The extension reads from and writes directly to the wiki's local file system (an `FSFileBackend` backend). It detects non-local backends (such as Amazon S3 or OpenStack Swift) and reports them, since in-place processing is not possible there without adaptation. The WebP destination directory (`images_webp/` by default) must be writable by the user PHP runs as.

2.4 PHP memory and execution time

- `memory_limit`: 256 MB minimum. A value of 128 MB works for most images but may fail on very large ones. *libvips markedly lowers this need (§16.5).*
- `max_execution_time`: not very critical, since the bulk of the work goes through the asynchronous job queue rather than direct web requests.

2.5 Installing the dependencies NEW

The extension runs with a bare minimum: an image library with WebP support. Every other dependency is **optional** with only a targeted benefit.

Dependency	Role	Required?
Imagick <i>or</i> GD (with WebP)	First pass + WebP encoding	Yes (either one)
libvips (<code>vips</code>)	Fast/light engine at scale (§4.7, §16)	No — opt-in
gifsicle	Lossless GIF first pass, animation-safe (§4.6)	No — GIF gain
zopfli	Second-pass lossless PNG, max compression (§4.4)	No — disk gain
oxipng	Second-pass lossless PNG, fast (§4.4)	No — disk gain
libheif+AV1 / <code>imageavif()</code>	Experimental AVIF (§4.9)	No — experimental

```
# Debian / Ubuntu – image library (one of) + optional binaries
sudo apt install php-imagick      # recommended
sudo apt install php-gd           # fallback, often already present
sudo apt install gifsicle libvips-tools zopfli
# RHEL / Alma / Rocky / Fedora (EPEL)
sudo dnf install php-imagick php-gd gifsicle vips-tools zopfli
# oxipng: often not packaged -> compile (avoids glibc issues)
cargo install oxipng
```

Check WebP support (mandatory):

```
php -r 'echo extension_loaded("imagick")&&in_array("WEBP",Imagick::queryFormats())?"Imagick+WebP OK\n":"";'
php -r '$i=gd_info();echo !empty($i["WebP Support"])?"GD+WebP OK\n":"GD without WebP\n";'
```

open_basedir trap (shared hosting / Plesk). If PHP is restricted, a binary in `/usr/bin` stays invisible to the web side even though it works on the command line. **Golden rule:** copy the binary *inside* the wiki tree (e.g. `httpdocs/bin/`), `chmod 755`, owner = the PHP user, then set its **absolute path**. This applies to `vips`, `gifsicle`, `zopfli` and `oxipng`.

3. Installation

3.1 Placing the files

Unpack the extension archive into the `extensions/` directory of your MediaWiki installation. The folder must be named exactly `VaultTecMediaOptimizer`, i.e. `extensions/VaultTecMediaOptimizer/`.

Important — FTP transfer. If you transfer the files over FTP, you must use **binary mode**, not ASCII mode. An ASCII-mode transfer corrupts the `extension.json` file in particular (it alters line endings and encoding), which causes the “is not a valid JSON file” error at startup. When in doubt, unpack the archive directly on the server over SSH.

3.2 Enabling in LocalSettings.php

```
wfLoadExtension( 'VaultTecMediaOptimizer' );
```

Any custom settings (see section 4) must be placed **after** this line, so they are not overwritten by the default values.

3.3 Database update

The extension creates two tables (`vtmo_image_optimization` and `vtmo_zopfli_thumb_stats`) and, on a version upgrade, adds a column. Run MediaWiki's update script:

```
php maintenance/run.php update --quick
```

On versions earlier than MediaWiki 1.40, the equivalent command is `php maintenance/update.php --quick`.

Note. The update script is idempotent: it can be re-run safely. It detects what already exists and applies only the necessary changes, whether this is a fresh install, an upgrade, or an already up-to-date database.

3.3.1 If blocked by another extension

If `update.php` fails because of another extension (one whose SQL file is not idempotent), the migration may never be reached. Either temporarily disable the offending extension, re-run `update.php`, then re-enable it; or create the objects manually via SQL:

```
ALTER TABLE vtmo_image_optimization
  ADD COLUMN IF NOT EXISTS io_png_zopfli_size INT UNSIGNED DEFAULT NULL;

CREATE TABLE IF NOT EXISTS vtmo_zopfli_thumb_stats (
  zts_id INT UNSIGNED NOT NULL,
  zts_thumb_count INT UNSIGNED DEFAULT 0 NOT NULL,
  zts_bytes_saved BIGINT UNSIGNED DEFAULT 0 NOT NULL,
  zts_updated_at BINARY(14) DEFAULT NULL,
  PRIMARY KEY(zts_id)
);
```

3.4 Post-install verification

Go to **Special:VTMOSatus**. All checks should be green (or amber for optional, non-blocking items). It is the reference tool for confirming the installation is healthy before launching the backfill.

4. Configuration

All settings are defined in `LocalSettings.php`, after the `wfLoadExtension` line. Each setting has a sensible default; only override the ones you want to change. The full prefix is `$wgVaultTecMediaOptimizer` (e.g. `$wgVaultTecMediaOptimizerEnabled`), abbreviated `...` in the tables.

4.1 General settings

Setting	Default	Description
<code>...Enabled</code>	<code>true</code>	Master switch. Setting it to <code>false</code> disables the extension entirely.
<code>...Formats</code>	<code>png, jpeg, gif</code>	MIME types to process. Array of strings (<code>image/png</code> , etc.).
<code>...MaxFileSize</code>	<code>52428800</code>	Maximum file size (in bytes) to process. 50 MiB by default.
<code>...StripMetadata</code>	<code>true</code>	Removes non-essential metadata (EXIF, etc.) while preserving color profile and transparency.

4.2 Original optimization

Setting	Default	Description
<code>...OptimizeOriginals</code>	<code>true</code>	Enables recompression of original files. If <code>false</code> , only WebP files are generated (originals untouched).

Original optimization depends on the type: PNG files are recompressed strictly losslessly; JPEG files are optimized (Huffman tables, progressive, metadata stripped) but via an Imagick re-encode that is not strictly lossless (the original quality is preserved so the loss is invisible); GIF files are optimized losslessly by `gifsicle` if enabled (\$4.6), otherwise left intact.

Note. Disabling `OptimizeOriginals` is useful if you do not want to modify the original files on disk. It does not affect WebP generation or the visitor experience, who always receive the lighter WebP versions.

4.3 WebP generation

Setting	Default	Description
<code>...WebPDirectory</code>	<code>images_webp</code>	Directory (relative to the wiki root) where WebP files are stored, separately from originals.
<code>...WebPQuality</code>	<code>85</code>	WebP quality for lossy images (JPEG). Recommended range: 80-90.
<code>...WebPLosslessForPng</code>	<code>true</code>	Generates PNG WebP files in lossless mode, preserving transparency.

4.4 Second-pass PNG recompression (zopflipng / oxipng)

Beyond the base optimization, the extension can apply a second pass of lossless PNG recompression using an external tool. Two engines are available. This second pass is **disabled by default**, and its only benefit is additional disk-space savings (on the order of 8 to 20% on PNGs). It does not improve the WebP files served to visitors: it is therefore only worthwhile if storage space is a concern. **zopflipng** = maximum compression but slow; **oxipng** = much faster (Rust, multi-threaded), nearly as small — recommended for bulk.

Setting	Default	Description
<code>...ZopfliEnabled</code>	<code>false</code>	Master switch for the second pass, regardless of engine.
<code>...PngEngine</code>	<code>zopflipng</code>	Engine: <code>zopflipng</code> or <code>oxipng</code> .
<code>...ZopfliBinary</code>	<code>zopflipng</code>	Path/command for the <code>zopflipng</code> binary.
<code>...ZopfliIterations</code>	<code>15</code>	<code>zopflipng</code> iterations. Higher = smaller but slower.
<code>...OxipngBinary</code>	<code>oxipng</code>	Path/command for the <code>oxipng</code> binary.
<code>...OxipngLevel</code>	<code>2</code>	<code>oxipng</code> level (0-6 or <code>max</code>). 2 is a good balance.

4.5 Backfill

Setting	Default	Description
<code>...ProcessThumbnails</code>	<code>true</code>	Also recompress thumbnails on the fly as they are generated.
<code>...BackfillBatchSize</code>	<code>20</code>	Files per batch when backfilling via the queue.
<code>...OnDemandThumbLimit</code>	<code>5</code>	Max missing thumbnail WebP files generated <i>synchronously</i> during a single render (cold-cache latency guard). <code>0</code> disables on-the-fly generation.

4.6 GIF optimization (gifsicle) NEW

A **strictly lossless, animation-safe** first GIF pass via the external `gifsicle` binary. The extension only ever passes `-O{level}`

(structural optimization): never resize, never alter pixels, timing or the loop counter. The rendered animation is identical pixel-for-pixel — only its on-disk representation shrinks. It runs on upload **and** during the CLI backfill; a missing or disabled `gifsicle` is a harmless no-op (WebP generation still proceeds). A keep-if-smaller guard prevents any growth.

Setting	Default	Description
...GifsicleEnabled	false	Enables lossless GIF optimization.
...GifsicleBinary	gifsicle	Path/command for the binary.
...GifsicleLevel	3	-0 level (1-3); 3 tries the most strategies.

4.7 libvips engine (optional) NEW

An alternative image engine based on the `vips` binary (no PHP extension needed). It produces a WebP of **identical quality/size** to Imagick/GD (same libwebp), but can be **lighter on memory**, especially on thumbnails (see §16.5 for honest measurements). If the binary is unusable, the extension **falls back automatically** to Imagick/GD.

Setting	Default	Description
...ImageEngine	auto	auto (Imagick then GD) or <code>vips</code> .
...VipsBinary	vips	Path/command for the <code>vips</code> binary.

4.8 Large-wiki mode: job queue NEW

Setting	Default	Description
... UseJobQueue	true	false = stop enqueueing on upload: optimize originals via the CLI (§8.6) on your own schedule. Thumbnails still get WebP on demand at render.

4.9 AVIF (experimental) EXPERIMENTAL

Generates an **AVIF** copy in addition to WebP and offers it *before* WebP inside `<picture>` (capable browsers pick AVIF, others fall back to WebP then the original). Depending on content and encoder, AVIF *can* be more compact than WebP at equal quality — but not always (on noise or at high quality it can be larger). The extension therefore **compares the AVIF to the WebP and keeps the AVIF only if strictly smaller**; otherwise it serves WebP. AVIF needs an **AV1** encoder (Imagick+libheif/AV1, PHP-GD `imageavif()`, or libvips `heifsave+AV1`); the extension **probes** this capability and falls back cleanly to WebP when absent. Slow encoding → off by default.

Setting	Default	Description
...AvifEnabled	false	Enables AVIF generation (in addition to WebP).
...AvifDirectory	images_avif	Directory for AVIF copies (sibling of <code>images/</code>).
...AvifQuality	50	AVIF quality (0-100). $\approx 45\text{-}55 \approx \text{WebP } 80\text{-}85$ visually (real gain is content/encoder-dependent). Alpha preserved.

5. Technical architecture

5.1 File processing flow

1. **Validation**: is the MIME type allowed? is the size under the limit? is the file writable?
2. **Original optimization** (if enabled): lossless recompression (PNG/JPEG; GIF via gifsicle, §4.6). Essential guard — the result replaces the original only if strictly smaller; otherwise the original is kept intact.
3. **Metadata refresh** (1.8.1): if the first pass actually shrank the original, MediaWiki's stored metadata (`img_sha1`, `img_size`, dimensions) is refreshed. Without this, duplicate detection and thumbnail cache invalidation would stay pinned to the pre-optimization bytes.
4. **WebP generation** (and AVIF if enabled) in the dedicated directory.
5. **Recording**: the sizes (original, optimized, WebP) are stored in the database for statistics.

Anti-bloat guard. Optimization always writes to a temporary file and replaces the original only if the new version is strictly smaller. This guarantees that an optimization can never increase a file's size — a classic pitfall when a re-encode produces a file larger than the source (common on PNGs already optimized by another tool). Writes are **atomic** (a unique temp file + `rename`), so a concurrent reader never sees a half-written file.

5.2 Delivery via the picture tag

On render (the *view* action), a hook rewrites image HTML to wrap it in a `<picture>` tag, offering the WebP source (and AVIF before it if enabled) with a fallback to the original. The browser automatically picks the best format it supports. High-density (Retina) screens are handled via `srcset` (1x, 1.5x, 2x).

Important. A source is emitted only if the corresponding WebP/AVIF file actually exists on disk. This is deliberate: unlike `<img srcset>`, a missing source in a `<picture>` does not fall back automatically and would show a broken image. If the WebP of a given size is unavailable, the original image is served intact. **Corollary (1.8.1 fix):** under GD, an **animated** GIF gets no WebP/AVIF (GD decodes only the first frame) — the animated GIF is kept so the animation is never frozen. Imagick and libvips produce animated WebP/AVIF and are unaffected.

5.3 WebP generation: on creation and on demand

- **On creation**: when MediaWiki generates a thumbnail, a hook immediately produces its WebP — for every new upload and every size requested for the first time.
- **On demand**: at render, if an image references a thumbnail whose WebP is missing but whose source exists on disk, the extension generates the missing WebP then, in the same flow that already serves the page (bounded by `OnDemandThumbLimit`).

This dual mechanism covers every size actually used, including thumbnails created before the extension was installed, or sizes MediaWiki does not regenerate (e.g. fixed infobox widths).

Performance. The very first render of a cold page may be slightly slower, as it generates the missing WebP files for the sizes it uses. Later renders are instant (the WebP is cached on disk). If generation fails repeatedly (e.g. a permissions problem on the WebP directory), watch the logs and check the destination directory's permissions.

5.4 Asynchronous processing

The backfill of existing files goes through MediaWiki's JobQueue, not web requests. Two job types are defined: standard image optimization and zopflipng/oxipng recompression. This decoupling avoids slowing pages down and spreads the load over time. In large-wiki mode (§4.8), the queue is bypassed in favour of CLI processing (§8.6).

5.5 Storage and cache

Tracking data lives in `vtmo_image_optimization` (one row per processed file). Aggregate statistics are cached via the object cache (`WANObjectCache`) with a short TTL, to avoid recomputing expensive sums on every view; the cache is invalidated on each write. Statistics queries are portable across MySQL/MariaDB, SQLite and PostgreSQL.

6. Special pages

The extension provides three special pages, accessible to administrators (right `vaulttecmediaoptimizer-admin`, granted by default to the `sysop` group). They have canonical English names and French aliases.

Role	French alias	Canonical name
Status / diagnostics	Spécial:VaultTec_État	Special:VTMOSStatus
Backfill	Spécial:VaultTec_Optimisation	Special:VTMOBackfill
Statistics	Spécial:VaultTec_Statistiques	Special:VTMOSStats

6.1 VTMOStatus (diagnostics)

This page checks the whole environment and shows each control with a colour code: green (compliant), amber (non-blocking warning), red (blocking problem), or neutral info. It covers PHP and MediaWiki versions, memory, image libraries and their WebP support, the **active engine** (Imagick/GD/vips) and its probe, the file system and directories, the database table, the second-pass PNG state (selected engine, shell execution, binary), the availability of **gifsicle** and **AVIF/AV1**, compatibility with other extensions, and the configuration. It is the first place to look when something behaves abnormally.

6.2 VTMOBackfill

This page drives the processing of files already present. It shows how many files remain and lets you schedule batches in the job queue. If the second-pass PNG is available, a dedicated button launches zopflipng/oxipng recompression of existing files. The button only appears if the engine is correctly detected.

6.3 VTMOStats

This page shows an overview of the gains achieved, computed from real on-disk sizes (before/after each step). It reports three savings figures (total / first pass / second pass).

Low gain if files were already optimized. The recorded “starting” size is the file's size when the extension first processed it, not the raw original. If your files were already optimized, little is left to gain and the “originals” percentage will be naturally low — not a failure, just clean files.

WebP costs disk, but saves bandwidth. It is normal and expected for the disk balance to be negative. WebP's goal is not to save server storage but to cut the weight of images sent to visitors (bandwidth). See §16 for a measured, honest figure.

Estimate, not measurement. The extension does not track real views. The reduction **percentage** is reliable (from real sizes), but the absolute volume depends on traffic and on browser/CDN caches that strongly reduce real traffic — treat it as an order of magnitude, not a bill (cf. §16).

The page also shows compression rates, a per-format breakdown, a second-pass PNG section, a per-engine breakdown, and the list of any failed files.

7. Installing the second-pass binaries

7.1 The `open_basedir` pitfall

Many hosts (notably under Plesk) restrict the paths PHP can read and execute via the `open_basedir` directive. Typically PHP only has access to the site directory and to `/tmp/`, but not to `/usr/bin/`. As a result, even if a binary is installed in `/usr/bin/` and works perfectly on the command line, PHP can neither see nor execute it from the web context — a very common and confusing cause of errors, since the binary “exists” yet remains unusable by the extension.

Golden rule. Always place the recompression binary inside the wiki tree (e.g. a `bin/` subfolder), so it is covered by `open_basedir`. Then configure the **full absolute path** to it, not a bare command name.

7.2 Installing oxipng (recommended)

oxipng ships as a precompiled binary. Beware, though: generic binaries from public repos may require a newer system library (glibc) than your server's, causing a “GLIBC_2.34 not found” error. The most reliable method is to compile oxipng on the machine itself (via Rust's cargo), then copy the resulting binary into the wiki tree.

```
mkdir -p /path/to/wiki/httpdocs/bin
cp /src/oxipng /path/to/wiki/httpdocs/bin/oxipng
chmod 755 /path/to/wiki/httpdocs/bin/oxipng
chown user:group /path/to/wiki/httpdocs/bin/oxipng
```

```
$wgVaultTecMediaOptimizerZopfliEnabled = true;
$wgVaultTecMediaOptimizerPngEngine = 'oxipng';
$wgVaultTecMediaOptimizerOxipngBinary = '/path/to/wiki/httpdocs/bin/oxipng';
```

7.3 Verifying shell execution on the web side

The extension needs PHP to be able to launch external processes (`proc_open` or `exec`). If those are in PHP's `disable_functions`, the second pass (and gifsicle/vips) is impossible. The VTMOStatus page reports shell execution **in the web context** — that is the context that matters, and it can differ from the command line.

8. Maintenance

8.1 Running the initial backfill

After installation, the files already present are not yet processed. Launch the backfill from **Special:VTMOBackfill**, which schedules batches in MediaWiki's job queue.

Prefer the command line for large volumes. The special page does not process files directly: it schedules jobs. By default MediaWiki runs those at the end of visitor requests (`$wgJobRunRate`). Since each optimization job launches an external tool that costs CPU time, the visitor's request waits for the job to finish — the wiki then looks slow, when in reality your visitors are doing the processing for you. For a bulk backfill or recompression, run the processing on the command line (SSH) instead, in a separate process that doesn't affect visitors.

```
screen -S vtmo-jobs
php maintenance/run.php runJobs.php --type VaultTecMediaOptimizerOptimizeImage
```

The `$wgJobRunRate` setting. Raising `$wgJobRunRate` (as some generic MediaWiki messages suggest) **worsens** the slowdown for visitors instead of easing it, since more heavy jobs run on their requests. For heavy processing the right value is **0** (no jobs on web requests), combined with a `runJobs.php` over SSH, or simply the dedicated CLI scripts (§8.6).

8.2 Repairing improperly inflated originals

`repairBloatedOriginals.php` repairs files whose “optimized” version is, due to a legacy defect, larger than the original. It recompresses each affected file with the configured PNG engine (oxipng recommended for speed), corrects the recorded sizes so statistics become consistent again, and refreshes MediaWiki's metadata. Always start with `--dry-run`; use the **absolute path** if you get “Script not found”.

```
php maintenance/run.php ../repairBloatedOriginals.php --dry-run
php maintenance/run.php ../repairBloatedOriginals.php --max=20
php maintenance/run.php ../repairBloatedOriginals.php
```

8.3 Recompressing with a second engine (oxipng then zopflipng)

Understand first. PNG is lossless, so each engine re-encodes from the decoded pixels. Chaining oxipng then zopflipng does **not** add up the two gains: the final result is determined by the last engine that writes. Concretely, “oxipng then zopflipng” yields the same result as “zopflipng alone”. The point of chaining is to deflate the bulk fast with oxipng, then run the slow engine only for finishing. A built-in guard keeps the second engine's output only if it is actually smaller — a file is never enlarged.

```
# first pass (configured engine, e.g. oxipng)
... recompressOriginals.php
# then set PngEngine = 'zopflipng' and re-run:
... recompressOriginals.php --reset
```

`--reset` re-marks already-processed PNGs as to-do so the new engine picks them up. `--dry-run` previews, `--max=N` limits. Thumbnails are not re-processed in bulk: purge them to force regeneration with the current engine.

8.4 Cache purge after message changes

```
php maintenance/run.php rebuildLocalisationCache --force
```

8.5 HTTP cache headers (recommendation)

To fully benefit from the generated WebP, configure your web server so WebP (and AVIF) files are cached for a long time by browsers — e.g. a long `Cache-Control` header on the `.webp` files under `images_webp/`. A major and often-forgotten lever to actually cut bandwidth (see §16).

8.6 CLI backfill — `optimizeImages.php` **NEW**

For large wikis, a dedicated script performs the backfill **directly on the command line** (first pass + WebP) **without the job queue** — ideal so visitors are not affected. It honours the configured engine (incl. vips). Combine it with large-wiki mode (§4.8):

```
// LocalSettings.php (large-wiki mode)
$wgVaultTecMediaOptimizerUseJobQueue = false;
```

```
screen -S vtmo
php maintenance/run.php ../optimizeImages.php --dry-run
php maintenance/run.php ../optimizeImages.php --batch=200
php maintenance/run.php ../optimizeImages.php --format=gif
```

Options: `--dry-run`, `--batch`, `--max`, `--start`, `--force`, `--format` (a subset of the configured formats), `--purge-queue`. The `purgeQueue.php` script empties the Optimize queue (and, with `--include-zopfli`, the second-pass queue).

One path at a time for a given corpus: either the queue (`runJobs.php`) or the CLI (`optimizeImages.php`). Don't run both at once on the same stock (atomic writes = no corruption, but duplicated work). Use `--purge-queue` if needed.

8.7 The thumbnail second pass is a job — process it off-line NEW

When MediaWiki generates a new PNG thumbnail, the extension immediately produces its WebP (fast, < 50 ms). The **zopflipng second pass** on that thumbnail, however — a pure disk gain — costs **seconds per file** (≈ 14 s, see §16). Running it during render would make the **visitor** pay that time: a cold gallery of 30 PNGs could add **minutes** to a page load. So this second pass is **not** run inline; it is handed to a **job** (`VaultTecMediaOptimizerThumbnailRecompress`) that recompresses the stored thumbnail in the background.

Why command-line generation is better. A job only helps if it is processed **off the visitor requests**. By default MediaWiki runs the queue at the end of a request (`$wgJobRunRate`), which pushes the heavy work back onto a visitor. The right setup is `$wgJobRunRate = 0`; plus a `runJobs.php` launched over **SSH/cron** (or the CLI backfill `optimizeImages.php`, §8.6). Then all heavy encoding runs on a dedicated server process, **never on a visitor's response time** — the same rule as for original optimization (§8.1). In **large-wiki mode** (`UseJobQueue = false`, §4.8), this job is not enqueued at all: the admin recompresses thumbnails on their own schedule.

9. Troubleshooting

9.1 “is not a valid JSON file” at startup

The `extension.json` file was corrupted, usually by an ASCII-mode FTP transfer. Re-transfer it in binary mode, or unpack the archive directly on the server.

9.2 The statistics page shows an unknown-column error

The database migration wasn't applied (missing column or table). Run `update.php` (section 3.3). If another extension blocks the updater, apply the migration manually (section 3.3.1).

9.3 The recompression binary is not found (although it exists)

The most common cause is `open_basedir` preventing PHP from accessing a binary outside the allowed tree (typically in `/usr/bin/`). Copy the binary into a wiki folder and configure its absolute path (section 7). Other causes: the wrong configuration variable (with `oxipng` it is `...OxipngBinary`, not the `zopfli` one) — and check the exact case. “GLIBC not found” = the precompiled binary needs a newer glibc; compile it on the machine.

9.4 Recompression is very slow

The `zopfli` engine is intrinsically slow, especially with a high iteration count on large files. Switch to `oxipng` (much faster), or lower the `zopfli` iterations (e.g. to 5). The final size is very close, for a much shorter time (see §16).

9.5 Statistics look inconsistent during processing

This is normal: during a backfill or repair, the values change on every refresh. Wait until processing finishes (“Progress” at 100%) before interpreting them. The WebP total may also fluctuate temporarily, as the metadata refresh invalidates some thumbnails that are then regenerated.

9.6 An image is served as PNG/JPEG instead of WebP

Reload the page once (the first render generates the missing WebP), then reload again. Check the WebP directory's write permissions and that `OnDemandThumbLimit` > 0. For an **animated GIF under GD**, the missing WebP is **intended** (§5.2) so as not to freeze the animation.

9.7 File permissions

Run maintenance scripts as the PHP user (often the same as the web server), never as `root`. Running as root creates root-owned files that PHP can no longer overwrite afterwards — a source of permission conflicts on later uploads.

10. Quick reference

10.1 Database tables

Table	Role
vtmo_image_optimization	One row per processed file: original/optimized/WebP sizes, status, etc.
vtmo_zopfli_thumb_stats	Aggregate thumbnail-recompression savings.

10.2 Access right

The three special pages require vaulttecmediaoptimizer-admin, granted by default to the sysop group.

10.3 Maintenance scripts

Script	Role and options
optimizeImages.php NEW	CLI backfill (first pass + WebP) without the queue; --dry-run/--batch/--max/--start/--force/--format/--purge-queue.
purgeQueue.php NEW	Empties the Optimize queue (--include-zopfli for the second pass); --dry-run.
recompressOriginals.php	Recompresses original PNGs with the configured engine; --reset to switch engines, --dry-run/--max/--batch.
repairBloatedOriginals.php	Repairs inflated originals; --dry-run/--max/--batch.

11. Frequently asked questions

Does the extension change my image quality?

Not for originals: PNG optimization is strictly lossless (identical pixels), and PNG WebP files are generated losslessly by default. JPEGs are re-encoded at high quality (85 by default), visually indistinguishable in the vast majority of cases. Tunable via `...WebPQuality`.

What do visitors whose browser doesn't support WebP see?

The original image, intact. The `<picture>` tag offers WebP first but always keeps the original `<img>` as fallback. All recent browsers support WebP; the fallback only concerns very old ones.

Must I enable the second pass (zopflipng/oxipng)?

No. The main benefit for visitors (lighter WebP) works without it. The second pass only adds disk-space savings on PNGs. Enable it if server storage is a concern.

What's the difference between zopflipng and oxipng?

Both recompress PNGs losslessly. zopflipng compresses a little more but is very slow; oxipng is much faster for a result within a few percent. For bulk processing, oxipng is recommended.

Can I change engine later?

Yes. Change `...PngEngine` at any time. Already-processed files are not reprocessed automatically, but any new processing (or a repair) uses the new engine.

Are animated GIFs preserved?

Yes. `gifsicle` (§4.6) optimization is strictly lossless and animation-safe; and under GD, animated GIFs never get a frozen WebP/AVIF (§5.2). Without `gifsicle`, the GIF original is not touched at all.

How much will I save?

It depends entirely on your media library. For visitors, WebP often cuts served image weight by 25–50% (see §16 for an honest, thumbnail-level figure). Disk savings on originals are more modest.

Does it work with cloud storage (S3, Swift)?

The extension is designed for a local file system (`FSFileBackend`). Non-local backends are detected and reported on the status page; object storage is not supported as-is.

An image looks broken after processing. What now?

Very rare thanks to the anti-bloat guard and the `<img>` fallback. If it happens, first check by accessing the thumbnail URL directly (it must return the image with an `image/png` type or equivalent). Then check the logs and section 9. Since the original is never deleted, no data is lost.

12. Uninstallation

The extension is designed to be removed cleanly, without leaving the wiki in a degraded state:

1. **Disable the extension:** remove (or comment out) the `wfLoadExtension( 'VaultTecMediaOptimizer' );` line and all `$wgVaultTecMediaOptimizer*` settings in `LocalSettings.php`.
2. **Purge the page cache:** pages then stop showing `<picture>` tags and revert to standard `<img>` tags pointing at the originals. Since the originals are intact, images display normally.
3. **(Optional) Delete the WebP files:** the `images_webp/` directory (and `images_avif/`) can be removed to reclaim space. It is no longer used once the extension is disabled.
4. **(Optional) Drop the tables:** if you won't reinstall, the tables can be dropped. Also remove the extension folder and the installed binaries.

Important. The original images are never deleted by the extension. After uninstall, your media library remains complete and functional. Only the (regenerable) WebP/AVIF versions disappear.

```
DROP TABLE IF EXISTS vtmo_zopfli_thumb_stats;  
DROP TABLE IF EXISTS vtmo_image_optimization;
```

13. Security

13.1 External binary execution

The external binaries (gifsicle, zopflipng, oxipng, vips) are invoked via `proc_open/exec` with arguments passed as an **array** (not a concatenated shell string), which avoids any shell interpretation and thus any command-injection risk. Binary paths come exclusively from the administrator's configuration, never from user input.

13.2 Path-traversal protection

On-demand WebP generation reads a thumbnail source and writes the corresponding WebP. To prevent a malicious filename from escaping the media tree, the extension rejects any name containing a traversal sequence (`..`), a path separator (`/` or `\`) or a NUL byte. Legitimate MediaWiki filenames never contain these characters.

13.3 Restriction to allowed types

Processing (optimization, WebP) is strictly limited to the configured MIME types (`image/png`, `image/jpeg`, `image/gif` by default). Any other file type is ignored. The maximum processed size is also capped by `...MaxFileSize`.

13.4 Access rights

The three special pages (status, backfill, statistics) are restricted to users holding the `vaulttecmidiaoptimizer-admin` right, granted by default only to the `sysop` group. Sensitive actions (launching a backfill, recompression) are therefore not accessible to ordinary users.

13.5 Harmlessness of generated markup

The HTML rewrite merely wraps existing `<img>` tags in a `<picture>` and adds a WebP/AVIF source. The image's original attributes are preserved as-is, and URLs are systematically encoded. No arbitrary data is injected into the HTML.

14. Glossary

Term	Definition
WebP	Modern image format with better compression than PNG and JPEG, lossy or lossless. Supported by all recent browsers.
AVIF	Often smaller than WebP on photos at equal quality (AV1 codec), but slower encoding and encoder-dependent. Experimental here (§4.9).
Lossless	Compression that preserves the exact original data: the decompressed image is strictly identical. The case of the extension's PNG and GIF/gifsicle optimization.
Lossy	Compression that drops information to reduce size, at the cost of slight alteration. Used for JPEG and, optionally, the WebP of JPEGs.
Thumbnail	A resized version of an image, generated by MediaWiki for display in pages (this is what is actually served).
picture tag	HTML element offering several image sources to the browser, which picks the most suitable. Used here to serve WebP/AVIF with a fallback to the original.
Backfill	Processing files already present at install time, as opposed to new uploads handled automatically.
Hook	A MediaWiki extension point letting an extension react to an event (upload, thumbnail generation, page render, etc.).
JobQueue	MediaWiki's mechanism for running work in the background, off web requests, so as not to slow page rendering.
libvips	A fast, memory-light image library/CLI (optional engine, §4.7).
gifsicle	An external lossless GIF optimizer, animation-safe (§4.6).
open_basedir	A PHP directive restricting accessible directories. A common cause of blocking for binaries outside the site tree.
zopflipng / oxipng	External lossless PNG recompression tools. zopflipng favours maximum compression, oxipng speed.

15. Step-by-step tutorial (from zero to production)

This tutorial walks, in order, through every step to install the extension, configure it, process an existing media library, and verify the result. The example paths (`/var/www/.../httpdocs`) are to be adapted to your installation.

Before you start. You need **SSH** access to your server and admin access to the wiki. With FTP only, some steps (running scripts, installing a binary) won't be possible: a shell is required for bulk processing.

Step 1 — Install the extension files

```
cd /var/www/vhosts/your-wiki/httpdocs/extensions
unzip /path/to/VaultTecMediaOptimizer.zip
ls VaultTecMediaOptimizer/extension.json # must exist
```

Over FTP: transfer in **binary mode** (ASCII corrupts `extension.json` → “is not a valid JSON file”).

Step 2 — Enable the extension

```
wfLoadExtension( 'VaultTecMediaOptimizer' );
```

Step 3 — Create the database tables

```
php maintenance/run.php update --quick
```

If it fails because of another extension, see section 3.3.1 to apply the migration manually.

Step 4 — Check status with the diagnostics page

Open **Special:VTMOSstatus**. The rows for PHP, MediaWiki, the image library and the file system must be green. The second-pass PNG section may be amber (not configured) — normal and non-blocking. Only proceed once the essential checks are green.

Step 5 — Understand what will be optimized

Type	Processing
PNG	Lossless on upload, further recompressible via oxipng/zopflipng. WebP generated.
JPEG	Optimized on upload (Imagick re-encode, quality preserved — not strictly lossless). WebP generated.
GIF	Lossless via <code>gifsicle</code> if enabled (§4.6), animation preserved; otherwise original untouched. WebP generated (animated under Imagick/libvips).

The JPEG case, transparently. JPEG original optimization goes through Imagick, which re-encodes the image: the Huffman-table step is lossless, but the re-encode itself is not 100% lossless. The extension preserves the original quality so the loss is invisible, and keeps the result only if smaller. If you require strictly lossless JPEG optimization, remove `image/jpeg` from `...Formats` (JPEG WebP will still be generated) and use a dedicated tool such as `jpegtran` outside the extension. Either way, visitors receive the WebP, unaffected by this nuance.

Step 6 — Run the backfill of existing files

New uploads are handled automatically, but files already present must be backfilled. Use **Special:VTMOBackfill** (queue) or, recommended for large wikis, the CLI (§8.6). Make sure the queue is processed, e.g. in a `screen`:

```
screen -S vtmo-jobs
php maintenance/run.php runJobs.php --type VaultTecMediaOptimizerOptimizeImage
```

Follow progress on **Special:VTMOStats**.

Step 7 (optional) — Install a PNG recompression engine

To save more disk on PNGs, install oxipng (fast) or zopflipng (slower). Optional, storage-only. See section 7, especially the `open_basedir` trap. This is also when to enable **gifsicle** (§4.6) and/or **libvips** (§4.7) if desired.

Step 8 (optional) — Recompress existing PNGs

```
screen -S vtmo-png
/opt/plesk/php/8.4/bin/php /abs/wiki/maintenance/run.php \
/abs/wiki/extensions/VaultTecMediaOptimizer/maintenance/recompressOriginals.php
```

Common pitfall — “Script not found”. Don't use a relative path: depending on the current directory, MediaWiki mis-concatenates paths and looks for the script under `maintenance/extensions/...`. Always give the **absolute path** of the script and of `run.php`.

For best compression, then switch to zopflipng and re-run with `--reset` (§8.3).

Step 9 — Verify WebP is served to visitors

```
<picture>
  <source srcset="/images_webp/...webp" type="image/webp">
  
</picture>
```

If an image is still served as PNG/JPEG, reload the page once (the first render generates the missing WebP), then reload again.

Step 10 — Purge caches at the end

Once the backfill and any recompression are done, purge the wiki page cache **and** the CDN, so pages reflect the final state.

Order matters. Purge **after** processing completes, not during. Purging mid-repair is counter-productive: thumbnails being invalidated would be re-cached in their old version.

Quick recap

1. Unpack into `extensions/` (binary FTP).
2. `wfLoadExtension( 'VaultTecMediaOptimizer' );`
3. `php maintenance/run.php update --quick.`
4. Check **Special:VTMOStatus** (all green).
5. Run the backfill (queue or CLI `optimizeImages.php`).
6. (Optional) install/enable gifsicle, libvips, oxipng.
7. (Optional) `recompressOriginals.php`, then zopflipng finishing with `--reset`.
8. Verify the `<picture>` tags on a few pages.
9. Purge caches (wiki + CDN) at the end.

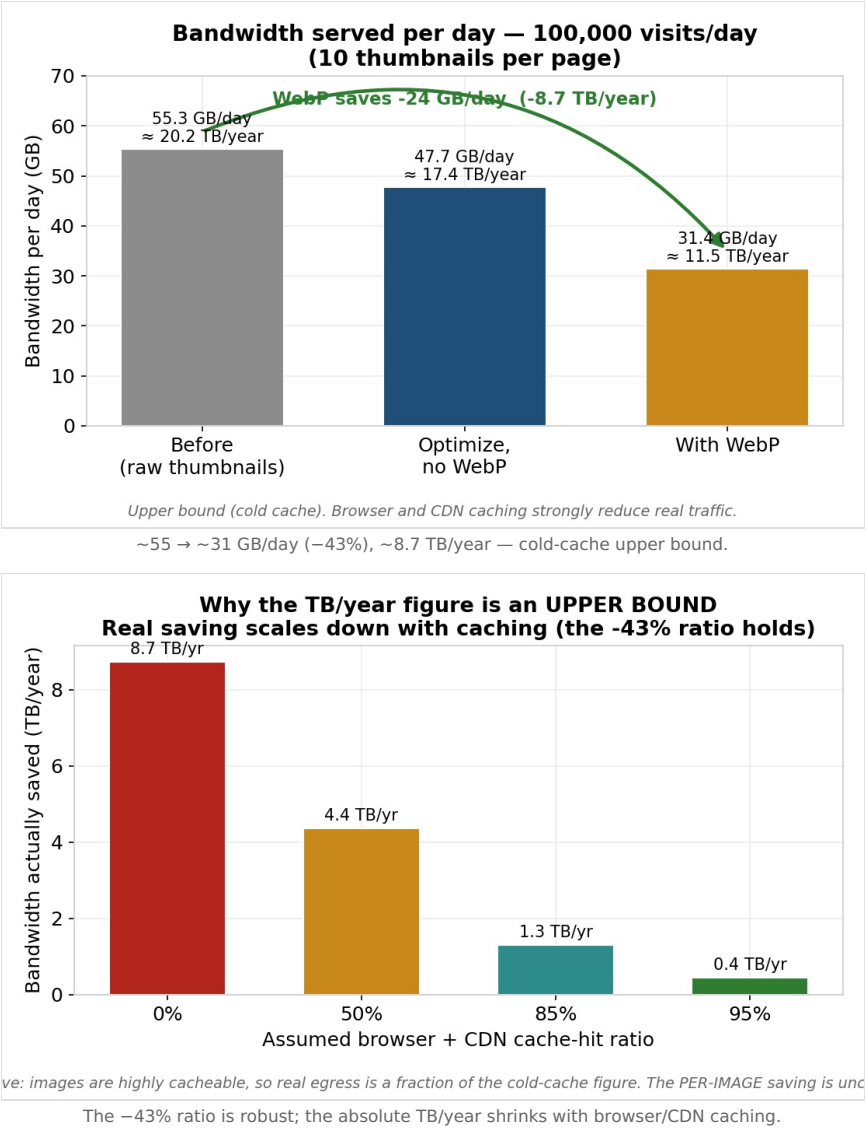
16. Benchmarks — cost / benefit NEW

Two families of figures. **(A)** a **cost/benefit model** of the real wiki, recomputed from explicit assumptions; **(B)** **real measurements** on the test box (PHP 8.4, libvips 8.15.1, ImageMagick 6.9.12, gifsicle 1.94, zopflipng, PHP-GD; **single core**). Reproducible via `docs/benchmarks/model.py` and `tests/integration/pipeline.php`.

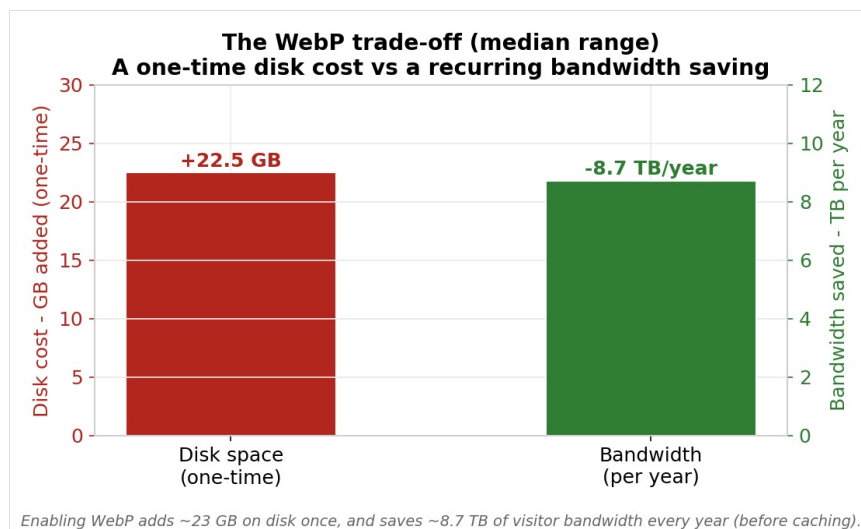
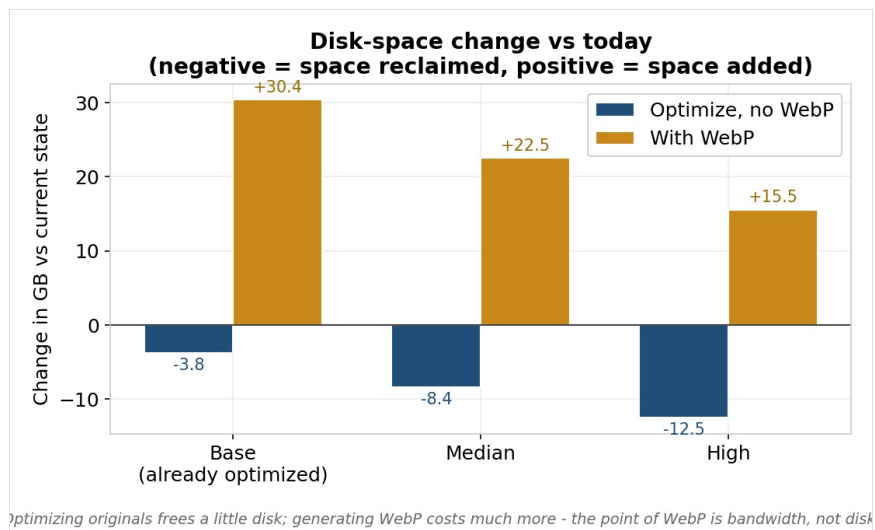
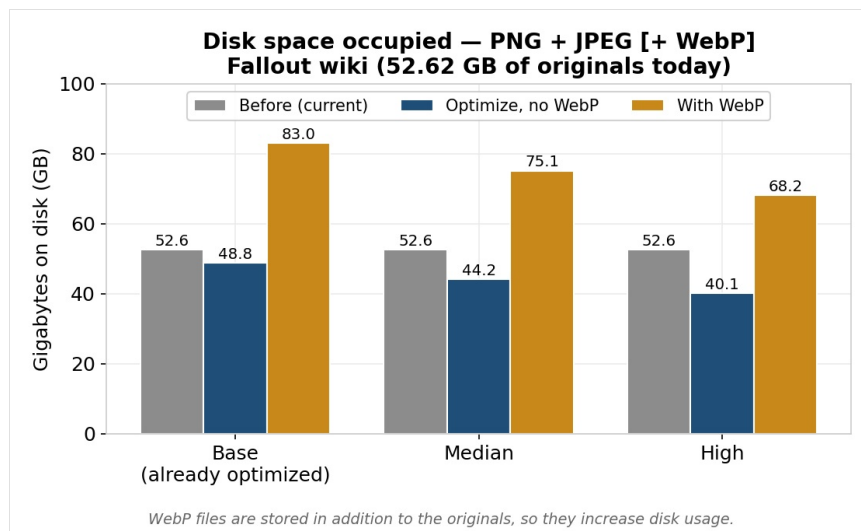
Model assumptions (decimal units; 365 d/yr): originals **52.62 GB** of PNG+JPEG; traffic **upper bound (cold cache)** of **100k views/day × 10 thumbnails**; average served thumbnail 55.3 KB; lossless thumbnail optimization −13.7%; WebP of thumbnail −34% ($\approx -43\%$ vs raw); original recompression −7/−16/−24% (low/median/high); WebP copies $\approx 70\%$ of the optimized original, stored **in addition**.

Honest framing. Pages serve **thumbnails** (already small), not originals, and WebP **adds** disk. We separate *served bandwidth* (the real benefit) from *per-file compression*.

16.1 Served bandwidth — the real benefit

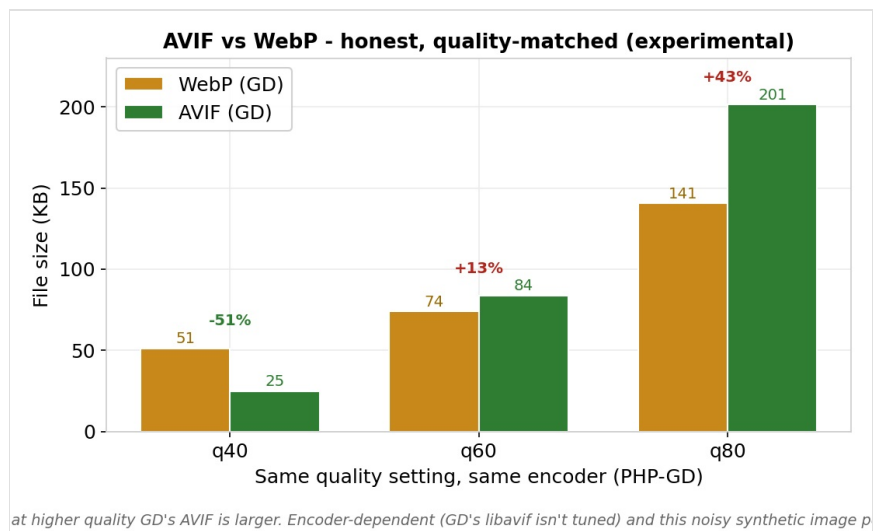
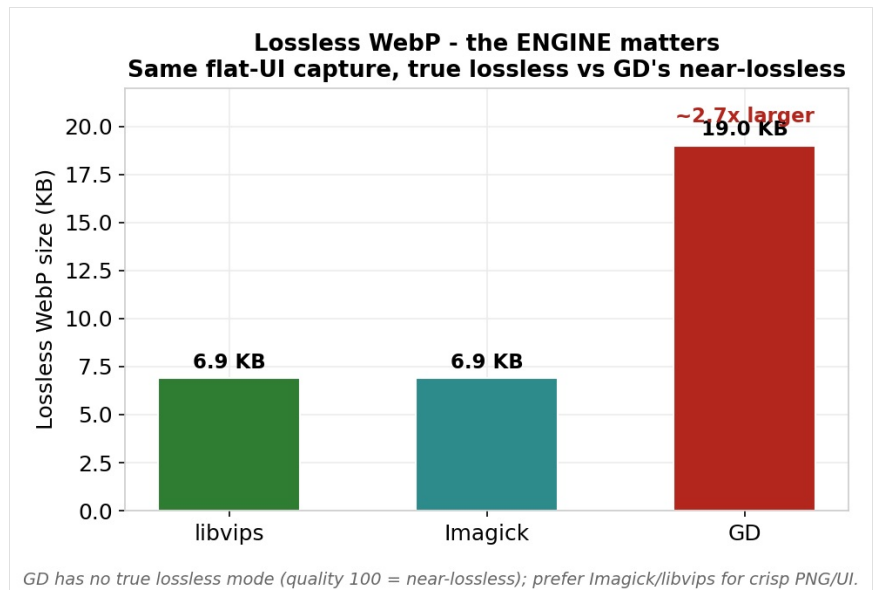
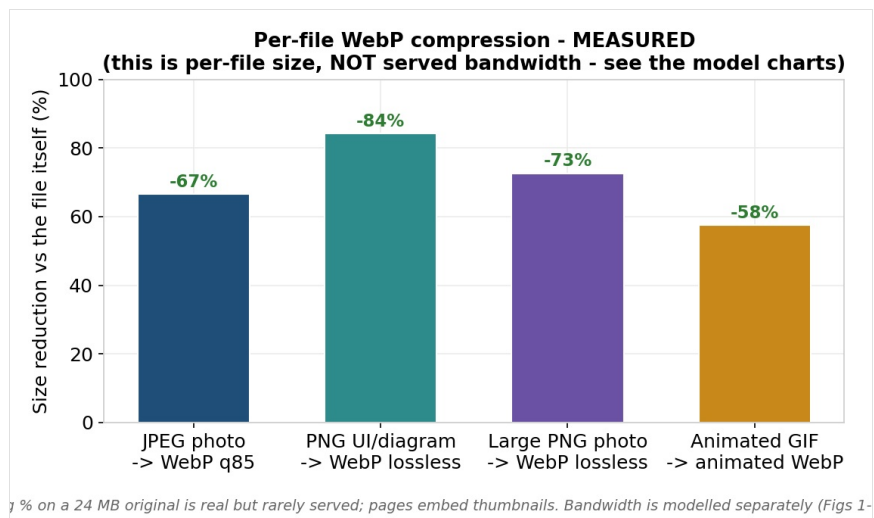


16.2 Disk: WebP adds to it



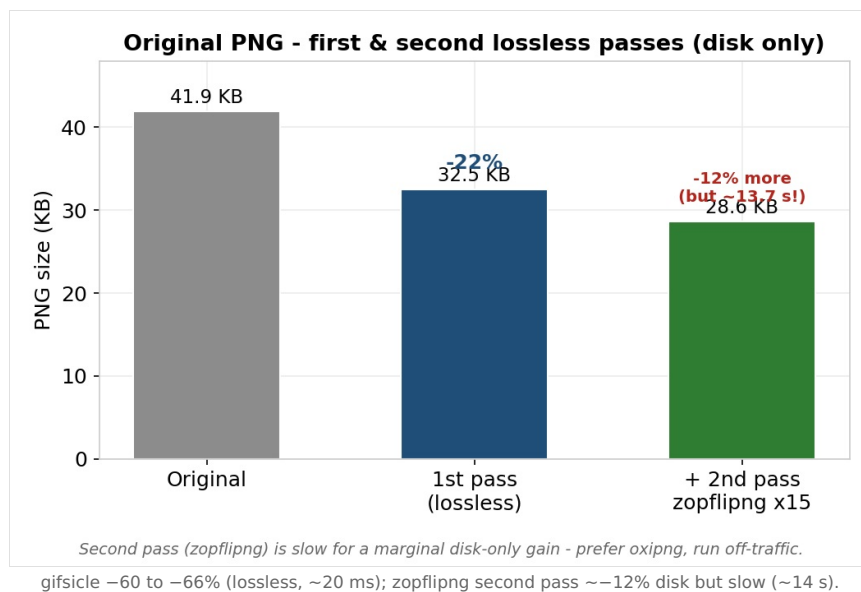
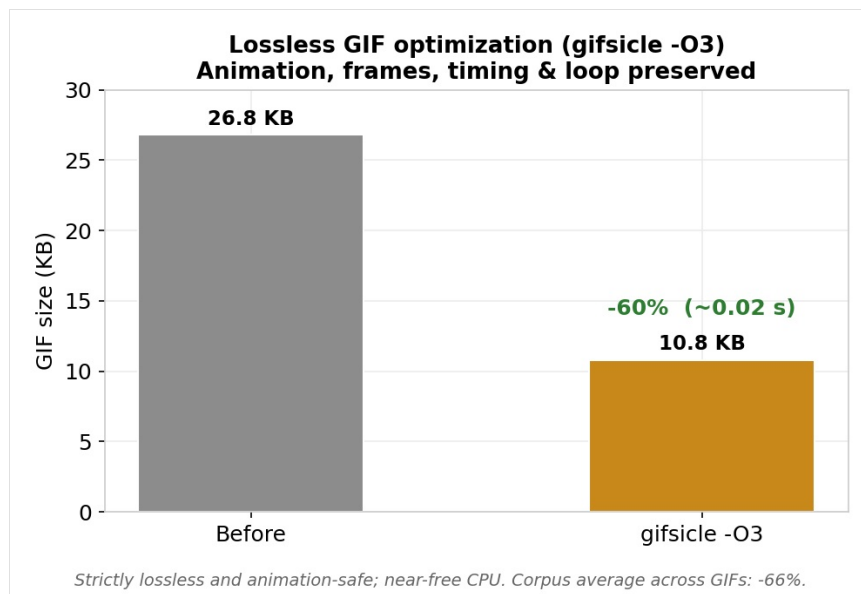
Disk goes up (52.6 → ~75 GB median); the trade-off is a one-time disk cost (~+22 GB) vs a recurring bandwidth saving.

16.3 Per-file compression (measured) — ≠ bandwidth

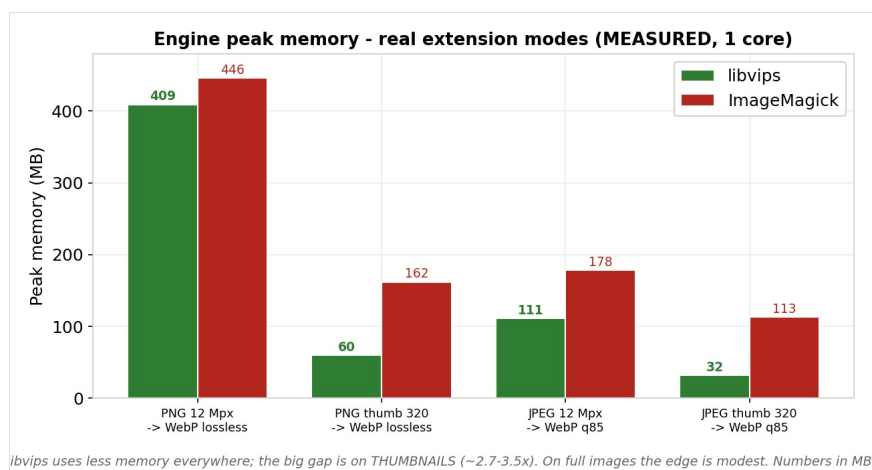


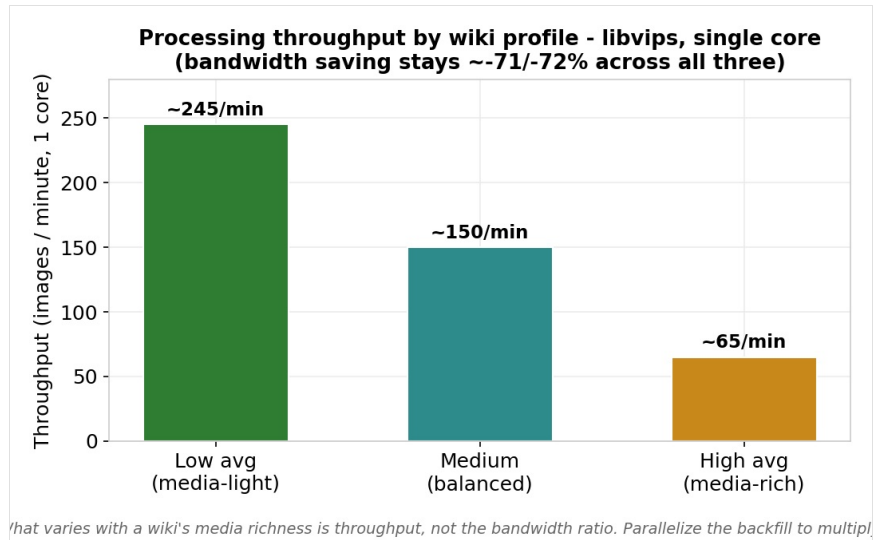
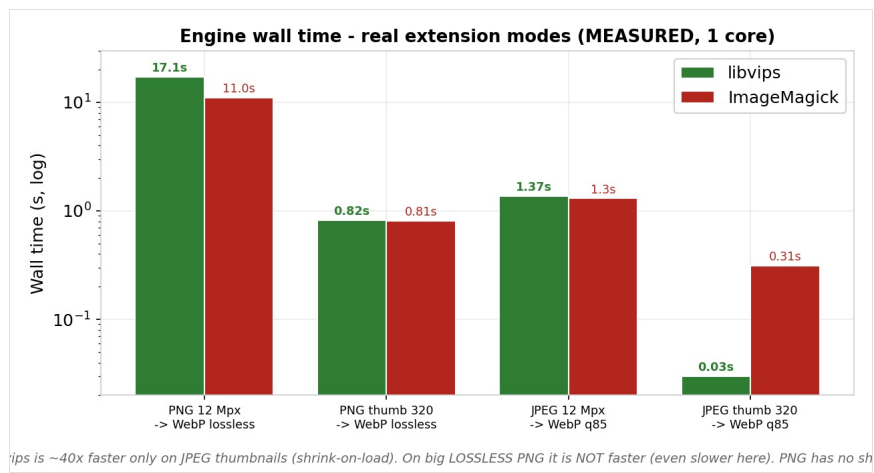
Per-file gains (not traffic); the engine matters (GD $\approx 2.7\times$ larger lossless); at equal quality AVIF is not systematically smaller (content/encoder-dependent).

16.4 GIF & 2nd-pass PNG (disk)



16.5 Engine: memory, speed, throughput





Measured at the extension's real modes (PNG→lossless WebP, JPEG→q85): libvips uses less memory everywhere (esp. thumbnails); ~40x faster only on JPEG thumbnails (shrink-on-load), NOT faster on big lossless PNG. Throughput ~245→~65 img/min by media richness.